

Turtle Graphics Project, Spring 2026

Tong Yi

Contents

1	CopyRight Warning	2
2	Example of Separating Declaration, Definition, and Test Code: FuelTank Class	2
2.1	FuelTank.hpp	2
2.2	FuelTank.cpp	3
2.3	FuelTankTest.cpp	4
3	Overview of Turtle Graphics Project	4
4	Task A: Implement Floor Class	5
4.1	Illustration of Floor Class	5
4.2	Download Floor.hpp	6
4.3	Implement Floor.cpp	7
4.4	Test Floor.cpp using FloorUnitTest.cpp	7
4.5	Submission for Task A	14
5	Task B: Implement Turtle Class	15
5.1	Illustration of Turtle Class	15
5.2	Download Turtle.hpp	15
5.3	Implement Turtle.cpp	19
5.4	Test Turtle.cpp using TurtleUnitTest.cpp	19
5.5	Submission for Task B	26
6	Task C: Implement TurtleGraphics Class	26
6.1	Illustration of TurtleGraphics Class	26
6.2	Download TurtleGraphics.hpp	26
6.3	Implement TurtleGraphics.cpp	26
6.4	Method run	26
6.4.1	Version 1: Reading Commands from the Console	27
6.4.2	Version 2: Reading Commands from a Two-Dimensional Vector	36
6.4.3	Comparison of Run Method Versions 1 and 2	37
6.4.4	Hint for Method <code>processCommand</code>	37
6.5	Download input3.txt	37
6.6	Test TurtleGraphics.cpp	37
6.7	Submission for Task C	38

7	Task D: Draw Shapes Side by Side	38
7.1	Drawing Digits of an Integer Side-by-Side	38
7.2	Drawing Characters of a String Side-by-Side	39
7.3	Skeleton Code for <code>drawSideBySide.cpp</code>	40
7.4	Compile and Run	61
7.5	Sample Input and Output	61
7.6	Submission for Task D	61
8	Makefile: a Tool for Project Build Management	61

1 Copyright Warning

1. This is copyrighted materials; you are not allowed to upload to the Internet.
2. Our project is different from similar products in Internet.
 - (a) Ask help only from teaching staff of this course.
 - (b) Use solutions from artificial intelligence like ChatGPT or online tutoring websites like, but not limited to, chegg.com, violates academic integrity and is not allowed.

2 Example of Separating Declaration, Definition, and Test Code: FuelTank Class

For robust software engineering practices, especially in large projects, the standard C++ approach is to divide the code into three distinct files:

Header File (.h or .hpp): Contains the declaration of the class (the class blueprint, member variables, and function prototypes).

Source File (.cpp): Contains the definitions (implementation logic) of the class’s member functions.

Test File (.cpp): Contains the `main()` function and the code dedicated to testing the functionality of the class.

This separation improves compile times (only changed implementation files need recompilation), enhances code organization, and facilitates code reuse across different parts of a project.

Code link: <https://onlinegdb.com/MzXcjYT2S->.

In online compilers like onlinegdb, the primary source file containing the `main` function must be named `main.cpp`. When working on your local computer, you have more flexibility and can name the test file something more descriptive, such as `FuelTankTest.cpp`. Naming the file `FuelTankTest.cpp` is generally preferred over `TestFuelTank.cpp` because it keeps related files grouped together when sorted alphabetically. This is a common practice in many C++ development environments to improve project organization and scannability.

2.1 FuelTank.hpp

```

1 #ifndef FUEL_TANK_H
2 #define FUEL_TANK_H

```

```

3 class FuelTank {
4 public:
5     FuelTank();
6     FuelTank(int startLevel);
7     int getGasLevel() const;
8     void fillGas(int numGallons);
9     void sendGas(int numGallons);
10
11 private:
12     int currGasLevel;
13 };
14 #endif

```

2.2 FuelTank.cpp

```

1 #include "FuelTank.hpp"
2
3 FuelTank::FuelTank() {
4     //int currGasLevel = 0; //wrong, with int before currGasLevel, the currGasLevel is
5     a local variable
6     //for default constructor,
7     //that statement does not initialize data member currGasLevel
8     currGasLevel = 0;
9 }
10
11 FuelTank::FuelTank(int startGasLevel) {
12     if (startGasLevel >= 0) {
13         currGasLevel = startGasLevel;
14     }
15     else {
16         currGasLevel = 0;
17     }
18 }
19
20 int FuelTank::getGasLevel() const {
21     return currGasLevel;
22 }
23
24 void FuelTank::sendGas(int gallons) {
25     if (currGasLevel >= gallons) {
26         currGasLevel -= gallons;
27     }
28 }
29
30 void FuelTank::fillGas(int gallons) {

```

```

30     currGasLevel += gallons;
31 }

```

2.3 FuelTankTest.cpp

```

1  //Sample output:
2  //gas level of an empty fuel tank is 0
3  //add 5 gallons to tank
4  //send 3 gallons out from tank
5  //current gas level is 2
6
7  #include "FuelTank.hpp"
8  #include <iostream>
9
10 int main() {
11     FuelTank tank(0);
12
13     std::cout << "gas level of an empty fuel tank is "
14               << tank.getGasLevel() << std::endl;
15
16     std::cout << "add 5 gallons to tank" << std::endl;
17     tank.fillGas(5);
18
19     std::cout << "send 3 gallons out from tank" << std::endl;
20     tank.sendGas(3);
21
22     std::cout << "current gas level is ";
23     std::cout << tank.getGasLevel() << std::endl;
24
25     return 0;
26 }

```

To run the code,

```

1  g++ -o run FuelTank.cpp FuelTankTest.cpp
2  ./run

```

3 Overview of Turtle Graphics Project

This project involves three classes: Floor, Turtle, and TurtleGraphics.

- Floor is a square-shape grid with characters ‘*’ and space character ‘ ’. There are operations like clear, mark, at to operate on the grid.
- Turtle lets a turtle lift up or put down a pen, turning left or right, and move.

- TurtleGraphics lets a turtle to draw character '*' on the floor by turning, moving, and lifting up or putting down a pen properly.

To run the project, we do the following.

1. Build a directory in your computer by entering the following command with return key in a terminal.

```
mkdir TurtleGraphics
```

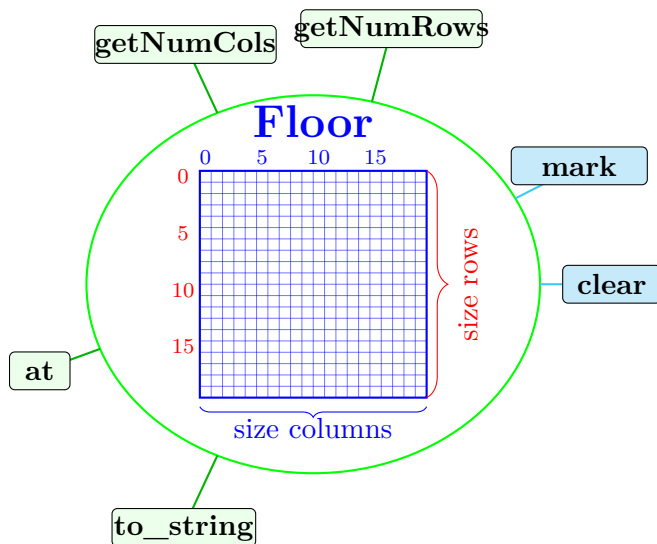
2. Move to the directory by pressing the following command with return key in a terminal.

```
cd TurtleGraphics
```

3. Download the required files and save them in the above directory.
4. You implement Floor, Turtle, and TurtleGraphics classes. Then test TurtleGraphics class. That is,
 - (a) Define Floor.cpp in Task A.
 - (b) Define Turtle.cpp in Task B.
 - (c) Define TurtleGraphics.cpp in Task C.
 - (d) Define drawSideBySide.cpp in Task D.

4 Task A: Implement Floor Class

4.1 Illustration of Floor Class



4.2 Download Floor.hpp

Download link: [Floor.hpp](#). Its contents are as follows.

```
1 #ifndef FLOOR_HPP
2 #define FLOOR_HPP
3
4 #include <vector>
5 #include <string>
6
7 /**
8  * @class Floor
9  * @brief Manages a 2D character grid with specific labeling and safety checks.
10  */
11 class Floor {
12 public:
13     /**
14      * @brief Default constructor.
15      * @brief Initializes a default 20 x 20 grid filled with spaces (' ').
16      */
17     Floor();
18
19     /**
20      * @brief Parameterized constructor.
21      * @param size The desired dimension for a square grid.
22      * @note If size < 10, a default 20x20 grid is created instead.
23      */
24     Floor(int size);
25
26     /** @return The number of rows in the grid. */
27     int getNumRows() const;
28
29     /** @return The number of columns in the grid. */
30     int getNumCols() const;
31
32     /**
33      * @brief Places a character at the specified coordinates.
34      * @param row The row index from 0 to getNumRows() - 1.
35      * @param col The column index from 0 to getNumCols() - 1.
36      * @param ch The character to place.
37      * @note If indices are out of bounds, the operation is silently ignored.
38      */
39     void mark(int row, int col, char ch);
40
41     /**
42      * @brief Retrieves the character at the specified coordinates.
43      * @param row The row index from 0 to getNumRows() - 1.
```

```

44     * @param col The column index from 0 to getNumCols() - 1.
45     * @return The character at (row, col), or the null character '\0' if out of
        bounds.
46     */
47     char at(int row, int col) const;
48
49     /**
50     * @brief Resets all cells in the grid to a space (' ').
51     */
52     void clear();
53
54     /**
55     * @brief Returns a string representation of the grid with coordinate labels.
56     *
57     * Labeling Rules:
58     * 1. Designed for grids up to 99x99.
59     * 2. The tens digit for row/column indices is displayed ONLY at multiples
60     *    of 10 (e.g., 10, 20, 30), appearing directly above or to the left.
61     * 3. The units digit is displayed for every row and column.
62     *
63     * @return A formatted string containing the labels and grid content.
64     */
65     std::string to_string() const;
66
67 private:
68     std::vector<std::vector<char>> grid;
69 };
70
71 #endif

```

4.3 Implement Floor.cpp

Read the requirement of Floor.hpp. Implement the constructors and methods in Floor.cpp.

4.4 Test Floor.cpp using FloorUnitTest.cpp

Download link: [FloorUnitTest.cpp](#) – its contents are as follows. The purpose is to test the constructors and methods of Floor.cpp.

```

1 //File name: FloorUnitTest.cpp
2 #include <iostream>
3 #include <vector>
4 #include <cstdlib> //srand
5 #include <ctime> //time
6 #include <sstream> //std::ostringstream
7 #include "Floor.hpp"

```

```

8 //g++ -o floor Floor.cpp FloorUnitTest.cpp
9 //test default constructor using
10 //./floor A
11
12 const int SIZE = 16;
13 Floor assignData(char data[][SIZE]);
14 void printCanvas(const Floor& canvas, const std::string& prompt);
15
16 int main(int argc, const char *argv[]) {
17     if (argc != 2) {
18         std::cout << "Need 'A'-'G' in command line parameter" << std::endl;
19         return -1;
20     }
21
22     //unit-testing for constructors and methods of Floor class
23     char type = *argv[1];
24     std::string prompt;
25
26     //IMPORTANT: If your output is inconsistent, temporarily replace srand(time(NULL))
27     //with srand(0).
28     srand(time(NULL));
29
30     //'A' for default constructor and
31     //'B' for non-default constructor
32     if (type == 'A' || type == 'B') {
33         if (type == 'A') {
34             prompt = "Call default constructor Floor canvas;";
35             Floor canvas;
36             printCanvas(canvas, prompt);
37
38             //Sample output:
39             //Call default constructor Floor canvas;
40             //num of rows: 20
41             //num of cols: 20
42             //contents of canvas is
43             // , , , , , , , , , , , , , , , , , , , , , ,
44             // , , , , , , , , , , , , , , , , , , , , , ,
45             // , , , , , , , , , , , , , , , , , , , , , ,
46             // , , , , , , , , , , , , , , , , , , , , , ,
47             // , , , , , , , , , , , , , , , , , , , , , ,
48             // , , , , , , , , , , , , , , , , , , , , , ,
49             // , , , , , , , , , , , , , , , , , , , , , ,
50             // , , , , , , , , , , , , , , , , , , , , , ,
51             // , , , , , , , , , , , , , , , , , , , , , ,
52             // , , , , , , , , , , , , , , , , , , , , , ,

```



```

53 // , , , , , , , , , , , , , , , , , , , , ,
54 // , , , , , , , , , , , , , , , , , , , , ,
55 // , , , , , , , , , , , , , , , , , , , , ,
56 // , , , , , , , , , , , , , , , , , , , , ,
57 // , , , , , , , , , , , , , , , , , , , , ,
58 // , , , , , , , , , , , , , , , , , , , , ,
59 // , , , , , , , , , , , , , , , , , , , , ,
60 // , , , , , , , , , , , , , , , , , , , , ,
61
62 }
63 else if (type == 'B') {
64     std::cout << "Enter size of the canvas (needs to be at least 10, otherwise,
65         convert it to 20): ";
66     int size;
67     std::cin >> size;
68     prompt = "Call non-default constructor Floor canvas(" + std::to_string(size)
69         + ");";
70     Floor canvas(size);
71     printCanvas(canvas, prompt);
72
73 //Enter size of the canvas (needs to be at least 10, otherwise, convert it to 20): 11
74 //Call non-default constructor Floor canvas(11);
75 //num of rows: 11
76 //num of cols: 11
77 //contents of canvas is
78 // , , , , , , , , , , ,
79 // , , , , , , , , , , ,
80 // , , , , , , , , , , ,
81 // , , , , , , , , , , ,
82 // , , , , , , , , , , ,
83 // , , , , , , , , , , ,
84 // , , , , , , , , , , ,
85 // , , , , , , , , , , ,
86 // , , , , , , , , , , ,
87 //
88
89 //do not take the parameter as it is,
90 //need to make sure that parameter size is >= 10
91 //Sample input/output:
92 //Enter size of the canvas (needs to be at least 10, otherwise, convert it to 20): 5
93 //Call non-default constructor Floor canvas(5);
94 //num of rows: 20
95 //num of cols: 20
96 //contents of canvas is

```



```

        space character\n";
143     isWrong = true; //set a tag
144     //break; //break only can break the inner loop, the outer loop still
        runs
145     }
146 }
147 }
148
149 if (!isWrong) {
150     std::cout << "clear method is correct\n";
151 }
152 else {
153     std::cout << "clear method is not correct\n";
154 }
155 //expected output:
156 //clear method is correct
157 }
158 else if (type == 'D') {
159     //test at method
160     bool isWrong = false;
161     for (int row = 0; row < canvas.getNumRows() && !isWrong; row++) {
162         for (int col = 0; col < canvas.getNumCols() && !isWrong; col++) {
163             if (canvas.at(row, col) != data[row][col]) {
164                 std::cout << "at method is not correct. The expected value is '" <<
                    data[row][col] << "'
165                 << ", however, the actual value is '" << canvas.at(row, col) << "
                    '\n";
166                 isWrong = true; //set a tag
167                 //break; //break only can break the inner loop, the outer loop
                    still runs
168             }
169         }
170     }
171
172     if (!isWrong) {
173         std::cout << "at method is correct\n";
174     }
175     else {
176         std::cout << "at method is not correct\n";
177     }
178 }
179 else if (type == 'E') {
180     //canvas.print(); //debug purpose
181
182     // build expected output manually from data[][]
183     std::ostringstream expected;

```

```

184
185 // print actual via stream
186 std::string actual = canvas.to_string();
187
188 // build expected string from data[][]
189 expected << " ";
190 for (int col = 0; col < SIZE; col++) {
191     if (col % 10 == 0 && col / 10 >= 1) {
192         expected << col / 10;
193     }
194     else {
195         expected << ' ';
196     }
197 }
198 expected << '\n';
199
200 expected << " ";
201 for (int col = 0; col < SIZE; col++) {
202     expected << col % 10;
203 }
204 expected << '\n';
205
206 for (int row = 0; row < SIZE; row++) {
207     expected << (row % 10 == 0 && row / 10 >= 1 ? char('0' + row / 10) : ' ');
208     expected << row % 10;
209     for (int col = 0; col < SIZE; col++) {
210         expected << data[row][col];
211     }
212     expected << '\n';
213 }
214
215 //std::cout << "'" << actual << "'\n\n";
216 //std::cout << "'" << expected.str() << "'\n\n";
217 if (actual == expected.str()) {
218     std::cout << "to_string method is correct\n";
219 }
220 else {
221     std::cout << "to_string method is not correct\n";
222 }
223 //expected output:
224 //to_string method is correct
225 }
226
227 else if (type == 'F') {
228     //test mark method of Floor class

```

```

229 Floor canvas(SIZE);
230 canvas.mark(0, 0, '*');          // top-left corner
231 canvas.mark(SIZE-1, SIZE-1, '*'); // bottom-right corner, minimum size is
    10
232 bool cornersCorrect = canvas.at(0,0) == '*' && canvas.at(SIZE-1, SIZE-1) == '*';
233
234 canvas.mark(-1, 0, '*');          // out of bounds, should do nothing
235 canvas.mark(0, SIZE, '*');        // out of bounds, should do nothing
236 canvas.mark(SIZE, 0, '*');        // out of bounds, should do nothing
237 canvas.mark(SIZE, SIZE, '*');     // out of bounds, should do nothing
238
239 // at() should return '\0' for invalid indices
240 bool outOfBoundsCorrect = canvas.at(-1,0) == '\0' &&
241     canvas.at(0,SIZE) == '\0' && canvas.at(SIZE,0) == '\0' && canvas.at(SIZE,SIZE)
    == '\0';
242
243 int row = SIZE / 2;
244 int col = SIZE / 2;
245 canvas.mark(SIZE/2, SIZE/2, '*');
246 bool centerCorrect = canvas.at(row, col) == '*';
247
248 row = rand() % SIZE;
249 col = rand() % SIZE;
250 canvas.mark(row, col, '*'); // ok even if (row,col) overlaps (0,0) or (SIZE-1,
    SIZE-1) since they are already '*'
251
252
253 bool randomCorrect = canvas.at(row, col) == '*';
254 if (cornersCorrect && outOfBoundsCorrect && centerCorrect && randomCorrect) {
255     std::cout << "mark method is correct\n";
256 }
257 else {
258     std::cout << "mark method is not correct\n";
259 }
260
261 //sample output:
262 //mark method is correct
263 }
264 else {
265     std::cout << "Unrecognized option: " << type << std::endl;
266     return -1;
267 }
268
269 return 0;
270 }
271

```

```

272 Floor assignData(char data[][SIZE]) {
273     Floor canvas(SIZE);
274
275     for (int row = 0; row < canvas.getNumRows(); row++) {
276         for (int col = 0; col < canvas.getNumCols(); col++) {
277             canvas.mark(row, col, data[row][col]); //set board[row][col] of *canvasPtr --
                a Floor object -- to be data[row][col]
278         }
279     }
280
281     return canvas;
282 }
283
284 void printCanvas(const Floor& canvas, const std::string& prompt) {
285     std::cout << prompt << '\n';
286     std::cout << "num of rows: " << canvas.getNumRows() << '\n';
287     std::cout << "num of cols: " << canvas.getNumCols() << '\n';
288     std::cout << "contents of canvas is\n";
289     for (int row = 0; row < canvas.getNumRows(); row++) {
290         for (int col = 0; col < canvas.getNumCols(); col++) {
291             std::cout << canvas.at(row, col) << ',';
292         }
293         std::cout << '\n';
294     }
295 }

```

Run the following commands.

```

1 g++ -o floor Floor.cpp FloorUnitTest.cpp

```

If the code compiles, you would get a floor executable file. To test the default constructor, run

```

1 ./floor A

```

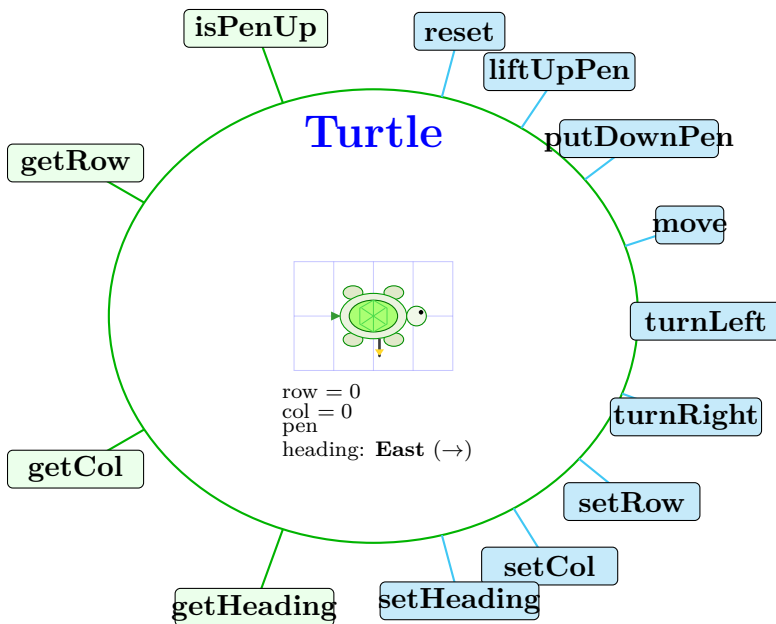
Test the rest of constructors and methods according to the comments in FloorUnitTest.cpp.

4.5 Submission for Task A

Submit Floor.cpp to gradescope.

5 Task B: Implement Turtle Class

5.1 Illustration of Turtle Class



5.2 Download Turtle.hpp

Download link: [Turtle.hpp](#) Its contents are as follows.

```
1 // File name: Turtle.hpp
2 #ifndef Turtle_HPP
3 #define Turtle_HPP
4
5 #include <ostream> //std::ostream, used in << operator for Direction
6
7 /**
8  * @brief Represents the state of the turtle's drawing pen.
9  *
10 * UP - the pen is lifted; movement does not draw.
11 * DOWN - the pen is lowered; movement draws a trail.
12 */
13 enum class Pen { UP, DOWN };
14
15 /**
16 * @brief Represents the four cardinal directions the turtle can face.
17 */
18 enum class Direction { NORTH, EAST, SOUTH, WEST };
19
20 /**
21 * @brief Overloads the stream insertion operator for Direction.
22 * @param os The output stream to write to.
```

```

23  * @param heading The Direction value to print.
24  * @return      Reference to the output stream, with the direction written
25  *              as a human-readable string: "NORTH", "EAST", "SOUTH", or "WEST".
26  */
27  std::ostream& operator<<(std::ostream& os, Direction heading);
28
29  /**
30  * @brief Models a turtle that moves on a 2-D grid and optionally draws a trail.
31  *
32  * The turtle tracks its position (row, col), the direction it faces (heading),
33  * and whether its pen is up or down. All data members are initialized at the
34  * point of declaration so no user-defined constructor is required.
35  */
36  class Turtle {
37  public:
38
39      // No user-defined constructor is needed.
40      // All data members are initialized where they are declared (modern C++ style),
41      // and the compiler-generated default constructor is sufficient.
42
43      /**
44       * @brief Resets the turtle to its initial state.
45       *
46       * After this call:
47       *   - pen      == Pen::UP
48       *   - heading == Direction::EAST
49       *   - row      == 0
50       *   - col      == 0
51       */
52      void reset();
53
54      /**
55       * @brief Reports whether the pen is currently lifted.
56       * @return true if pen == Pen::UP (turtle does not draw while moving).
57       * @return false if pen == Pen::DOWN (turtle draws while moving).
58       */
59      bool isPenUp() const;
60
61      /**
62       * @brief Lifts the pen so that subsequent moves do not draw.
63       * @post pen == Pen::UP
64       */
65      void liftUpPen();
66
67      /**
68       * @brief Lowers the pen so that subsequent moves draw a trail.

```



```

69  * @post pen == Pen::DOWN
70  */
71  void putDownPen();
72
73  /**
74   * @brief Moves the turtle forward by the given number of steps in its current
       heading.
75   *
76   * The axis updated depends on the current heading:
77   *   - EAST : col += steps
78   *   - WEST : col -= steps
79   *   - SOUTH : row += steps
80   *   - NORTH : row -= steps
81   *
82   * Negative values for steps move the turtle in the opposite direction.
83   * No bounds checking is performed; the caller is responsible for
84   * keeping the turtle within any grid boundaries.
85   *
86   * @param steps Number of grid cells to move (may be negative).
87   */
88  void move(int steps);
89
90  /**
91   * @brief Rotates the turtle 90 degrees to the left (counter-clockwise).
92   *
93   * Heading transitions:
94   *   EAST -> NORTH
95   *   NORTH -> WEST
96   *   WEST -> SOUTH
97   *   SOUTH -> EAST
98   *
99   * @post heading is updated to the next counter-clockwise direction.
100  */
101  void turnLeft();
102
103  /**
104   * @brief Rotates the turtle 90 degrees to the right (clockwise).
105   *
106   * Heading transitions:
107   *   EAST -> SOUTH
108   *   SOUTH -> WEST
109   *   WEST -> NORTH
110   *   NORTH -> EAST
111   *
112   * @post heading is updated to the next clockwise direction.
113   */

```

```

114 void turnRight();
115
116 /**
117  * @brief Sets the turtle's row position on the grid.
118  *
119  * No bounds checking is performed; the caller is responsible for
120  * passing a valid row index.
121  *
122  * @param row The new row index (0-based).
123  */
124 void setRow(int row);
125
126 /**
127  * @brief Sets the turtle's column position on the grid.
128  *
129  * No bounds checking is performed; the caller is responsible for
130  * passing a valid column index.
131  *
132  * @param col The new column index (0-based).
133  */
134 void setCol(int col);
135
136 /**
137  * @brief Returns the turtle's current row position.
138  * @return The current row index (0-based).
139  */
140 int getRow() const;
141
142 /**
143  * @brief Returns the turtle's current column position.
144  * @return The current column index (0-based).
145  */
146 int getCol() const;
147
148 /**
149  * @brief Sets the direction the turtle is facing.
150  *
151  * Because Direction is a scoped enum, only the four defined values
152  * (NORTH, EAST, SOUTH, WEST) are valid; no explicit validation is needed.
153  *
154  * @param heading The new heading for the turtle.
155  */
156 void setHeading(Direction heading);
157
158 /**
159  * @brief Returns the direction the turtle is currently facing.

```

```

160     * @return The current heading as a Direction value.
161     */
162     Direction getHeading() const;
163
164 private:
165     // Modern C++ preferred style: initialize members where they are declared,
166     // and let the compiler generate the default constructor.
167     Pen      pen      = Pen::UP;          ///< Current pen state (UP or DOWN).
168     Direction heading = Direction::EAST;   ///< Current heading (NORTH/EAST/SOUTH/WEST).
169     int      row      = 0;                ///< Current row position on the grid (0-based).
170     int      col      = 0;                ///< Current column position on the grid (0-based
171     );
172 };
173 #endif

```

5.3 Implement Turtle.cpp

Read the requirements in `Turtle.hpp`. Implement all methods declared in `Turtle.hpp` inside `Turtle.cpp`.
For your convenience, here are the code to overload `>>` operator for enum `Direction` class.

```

1 std::ostream& operator<<(std::ostream& os, Direction heading) {
2     switch (heading) {
3         case Direction::EAST: return os << "EAST";
4         case Direction::WEST: return os << "WEST";
5         case Direction::SOUTH: return os << "SOUTH";
6         case Direction::NORTH: return os << "NORTH";
7         default:              return os << "Unknown";
8     }
9 }

```

5.4 Test Turtle.cpp using TurtleUnitTest.cpp

Download link: [TurtleUnitTest.cpp](#) Its contents are as follows.

```

1 //file name: TurtleUnitTest.cpp
2 #include <iostream>
3 #include <cstdlib> //srand
4 #include <ctime> //time
5 #include "Turtle.hpp"
6 //g++ -o turtle Turtle.cpp TurtleUnitTest.cpp
7 //test default constructor using
8 //./turtle A
9
10 int main(int argc, const char *argv[]) {
11     if (argc != 2) {

```

```

12     std::cout << "Need 'A'-'G' in command line parameter" << std::endl;
13     return -1;
14 }
15
16 //unit-testing for methods of Turtle class, which has no constructors
17 char type = *argv[1];
18 std::string prompt;
19
20 srand(time(NULL));
21 const int SIZE = 20;
22 switch (type) {
23     case 'A': { //test reset method
24         Turtle tur;
25         tur.reset();
26         if (tur.isPenUp() && tur.getRow() == 0 && tur.getCol() == 0 && tur.getHeading() ==
            Direction::EAST) {
27             std::cout << "reset method is correct" << std::endl;
28         }
29         else {
30             std::cout << "reset method is not correct" << std::endl;
31         }
32         break;
33     }
34     //When running ./turtle A,
35     //where turtle obtained by
36     //g++ -o turtle TurtleUnitTest.cpp Turtle.cpp
37     //expected output is as follows
38     //reset method is correct
39
40
41     case 'B': { //test isPenUp, liftUpPen, and putDownPen methods
42         Turtle tur;
43         tur.liftUpPen();
44         bool penUpTrue = tur.isPenUp();
45
46         tur.putDownPen();
47         bool penUpFalse = tur.isPenUp();
48
49         if (penUpTrue && penUpFalse == false) {
50             std::cout << "isPenUp, liftUpPen, and putDownPen methods are correct" << std::
                endl;
51         }
52         else {
53             std::cout << "isPenUp, liftUpPen, and putDownPen methods are not correct" << std
                ::endl;
54         }

```

```

55
56     break;
57 }
58 //Running the following two comands
59 //g++ -o turtle TurtleUnitTest.cpp Turtle.cpp
60 ./turtle B
61 //expected output: isPenUp, liftUpPen, and putDownPen methods are correct
62
63     case 'C': { //test setRow and getRow methods
64         Turtle tur;
65         int row = rand() % SIZE;
66         tur.setRow(row);
67         if (tur.getRow() == row) {
68             std::cout << "setRow and getRow methods are correct" << std::endl;
69         }
70         else {
71             std::cout << "setRow and getRow methods are not correct" << std::endl;
72         }
73
74         break;
75     }
76 //expected output after running ./turtle C
77 //setRow and getRow methods are correct
78
79     case 'D': { //test setCol and getCol methods
80         Turtle tur;
81         int col = rand() % SIZE;
82         tur.setCol(col);
83         if (tur.getCol() == col) {
84             std::cout << "setCol and getCol methods are correct" << std::endl;
85         }
86         else {
87             std::cout << "setCol and getCol methods are not correct" << std::endl;
88         }
89
90         break;
91     }
92 //expected output after running ./turtle D
93 //setCol and getCol methods are correct
94
95     case 'E': { //test turnLeft, setHeading and getHeading methods
96         Turtle tur;
97
98         tur.setHeading(Direction::EAST);
99         tur.turnLeft();
100        if (tur.getHeading() != Direction::NORTH) {

```

```

101     std::cout << "When direction is EAST, turnLeft method is not correct." << std::
102         endl;
103     break;
104 }
105
106 tur.setHeading(Direction::WEST);
107 tur.turnLeft();
108 if (tur.getHeading() != Direction::SOUTH) {
109     std::cout << "When direction is WEST, turnLeft method is not correct." << std::
110         endl;
111     break;
112 }
113
114 tur.setHeading(Direction::NORTH);
115 tur.turnLeft();
116 if (tur.getHeading() != Direction::WEST) {
117     std::cout << "When direction is NORTH, turnLeft method is not correct." << std::
118         endl;
119     break;
120 }
121
122 tur.setHeading(Direction::SOUTH);
123 tur.turnLeft();
124 if (tur.getHeading() != Direction::EAST) {
125     std::cout << "When direction is SOUTH, turnLeft method is not correct." << std::
126         endl;
127     break;
128 }
129
130 std::cout << "turnLeft method is correct." << std::endl;
131 break;
132 }
133
134 //expected output after running ./turtle E:
135 //turnLeft method is correct.
136
137 case 'F': { //test turnRight, setHeading and getHeading methods
138     Turtle tur;
139
140     tur.setHeading(Direction::EAST);
141     tur.turnRight();
142     if (tur.getHeading() != Direction::SOUTH) {
143         std::cout << "When direction is EAST, turnRight method is not correct." << std::
144             endl;
145         break;
146     }
147 }

```

```

142 tur.setHeading(Direction::WEST);
143 tur.turnRight();
144 if (tur.getHeading() != Direction::NORTH) {
145     std::cout << "When direction is WEST, turnRight method is not correct." << std::
        endl;
146     break;
147 }
148
149 tur.setHeading(Direction::NORTH);
150 tur.turnRight();
151 if (tur.getHeading() != Direction::EAST) {
152     std::cout << "When direction is NORTH, turnRight method is not correct." << std
        ::endl;
153     break;
154 }
155
156 tur.setHeading(Direction::SOUTH);
157 tur.turnRight();
158 if (tur.getHeading() != Direction::WEST) {
159     std::cout << "When direction is SOUTH, turnRight method is not correct." << std
        ::endl;
160     break;
161 }
162
163 std::cout << "turnRight method is correct." << std::endl;
164 break;
165 }
166 //expected output after running ./turtle F:
167 //turnRight method is correct.
168
169 case 'G': { // void move(int steps);
170     Turtle tur;
171
172     //move in east direction
173     tur.setHeading(Direction::EAST);
174     int numStepsEast = 0;
175     while (numStepsEast == 0) {
176         numStepsEast = rand() % SIZE;
177     }
178     //Make sure numStepsEast is not zero.
179
180     tur.move(numStepsEast);
181
182     if (tur.getRow() != 0 || tur.getCol() != numStepsEast) {
183         std::cout << "When direction is Direction::EAST, move method of " <<
            numStepsEast << " steps is wrong." << std::endl;

```

```

184     break;
185 }
186
187 //move in south direction from (0, numStepsEast)
188 tur.setHeading(Direction::SOUTH);
189 int numStepsSouth = 0;
190 while (numStepsSouth == 0) {
191     numStepsSouth = rand() % SIZE;
192 }
193 //Make sure numStepsSouth is not zero.
194
195 tur.move(numStepsSouth);
196 if (tur.getRow() != numStepsSouth || tur.getCol() != numStepsEast) {
197     std::cout << "When direction is Direction::SOUTH, move method of " <<
198         numStepsSouth << " steps is wrong." << std::endl;
199     break;
200 }
201
202 //set tur to the rightmost position of the first row, where row index is 0 and
203     column index is SIZE-1.
204 tur.setRow(0);
205 tur.setCol(SIZE-1);
206
207 //move in west direction
208 tur.setHeading(Direction::WEST);
209
210 int numStepsWest = 0;
211 while (numStepsWest == 0) {
212     numStepsWest = rand() % SIZE;
213 }
214 //Make sure numStepsWest is not zero.
215
216 tur.move(numStepsWest);
217 if (tur.getRow() != 0 || tur.getCol() != SIZE -1 -numStepsWest) {
218     std::cout << "When direction is Direction::WEST, move method of " <<
219         numStepsWest << " steps is wrong." << std::endl;
220     break;
221 }
222
223 //set tur to the bottom right position in the grid, where row index is SIZE-1
224     and column index is SIZE-1.
225 tur.setRow(SIZE-1);
226 tur.setCol(SIZE-1);
227
228 //move in north direction
229 tur.setHeading(Direction::NORTH);

```



```

226     int numStepsNorth = 0;
227     while (numStepsNorth == 0) {
228         numStepsNorth = rand() % SIZE;
229     }
230     //Make sure numStepsNorth is not zero.
231
232     tur.move(numStepsNorth);
233     if (tur.getRow() != SIZE -1 -numStepsNorth || tur.getCol() != SIZE -1) {
234         std::cout << "When direction is Direction::NORTH, move method of " <<
            numStepsNorth << " steps is wrong." << std::endl;
235         break;
236     }
237
238     std::cout << "move method is correct" << std::endl;
239
240     break;
241 }
242 //expected output after running ./turtle G:
243 //move method is correct
244
245     default:
246         std::cout << "Unrecognized option: " << type << std::endl;
247 }
248
249 return 0;
250 }

```

Compile and link Turtle.cpp and TurtleUnitTest.cpp using the following command.

```
g++ -o turtle Turtle.cpp TurtleUnitTest.cpp
```

Run each test case by passing the corresponding letter as a command-line argument. For example, to test the reset method, run

```
./turtle A
```

Test cases A through G correspond to the following methods:

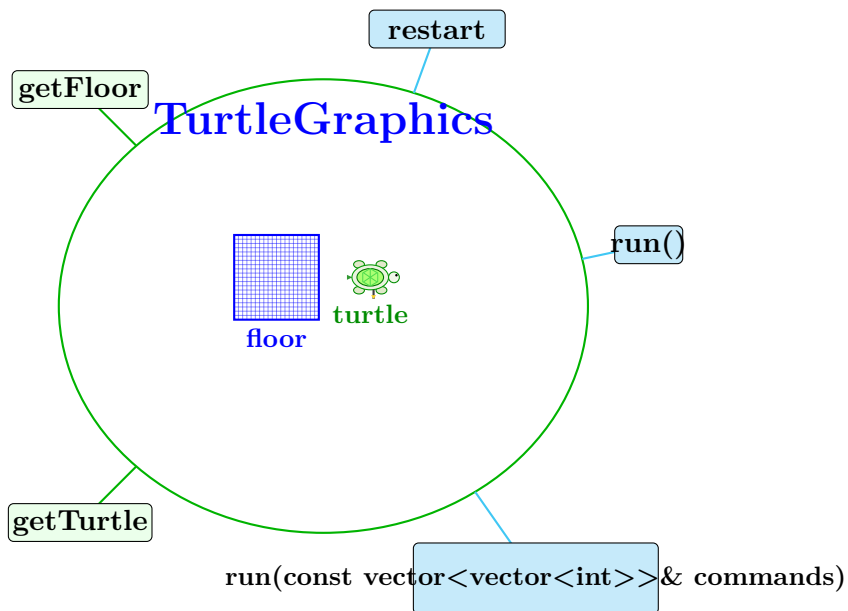
- A — reset
- B — isPenUp, liftUpPen, putDownPen
- C — setRow, getRow
- D — setCol, getCol
- E — turnLeft, setHeading, getHeading
- F — turnRight, setHeading, getHeading
- G — move

5.5 Submission for Task B

Submit `Turtle.cpp` to gradescope.

6 Task C: Implement TurtleGraphics Class

6.1 Illustration of TurtleGraphics Class



6.2 Download TurtleGraphics.hpp

Download link: [TurtleGraphics.hpp](#)

6.3 Implement TurtleGraphics.cpp

Read the requirements in `TurtleGraphics.hpp`. Implement all methods declared in `TurtleGraphics.hpp` inside `TurtleGraphics.cpp`.

6.4 Method run

There are two versions of `run` method in `TurtleGraphics.cpp`.

```
1 // Runs the turtle graphics program in interactive mode.
2 // Reads commands from the console (stdin) until the user
3 // enters 9 to stop. Calls restart() before processing commands.
4 // Valid commands:
5 // 1 - pen up
6 // 2 - pen down
7 // 3 - turn right
8 // 4 - turn left
9 // 5 - move (followed by number of steps)
```

```

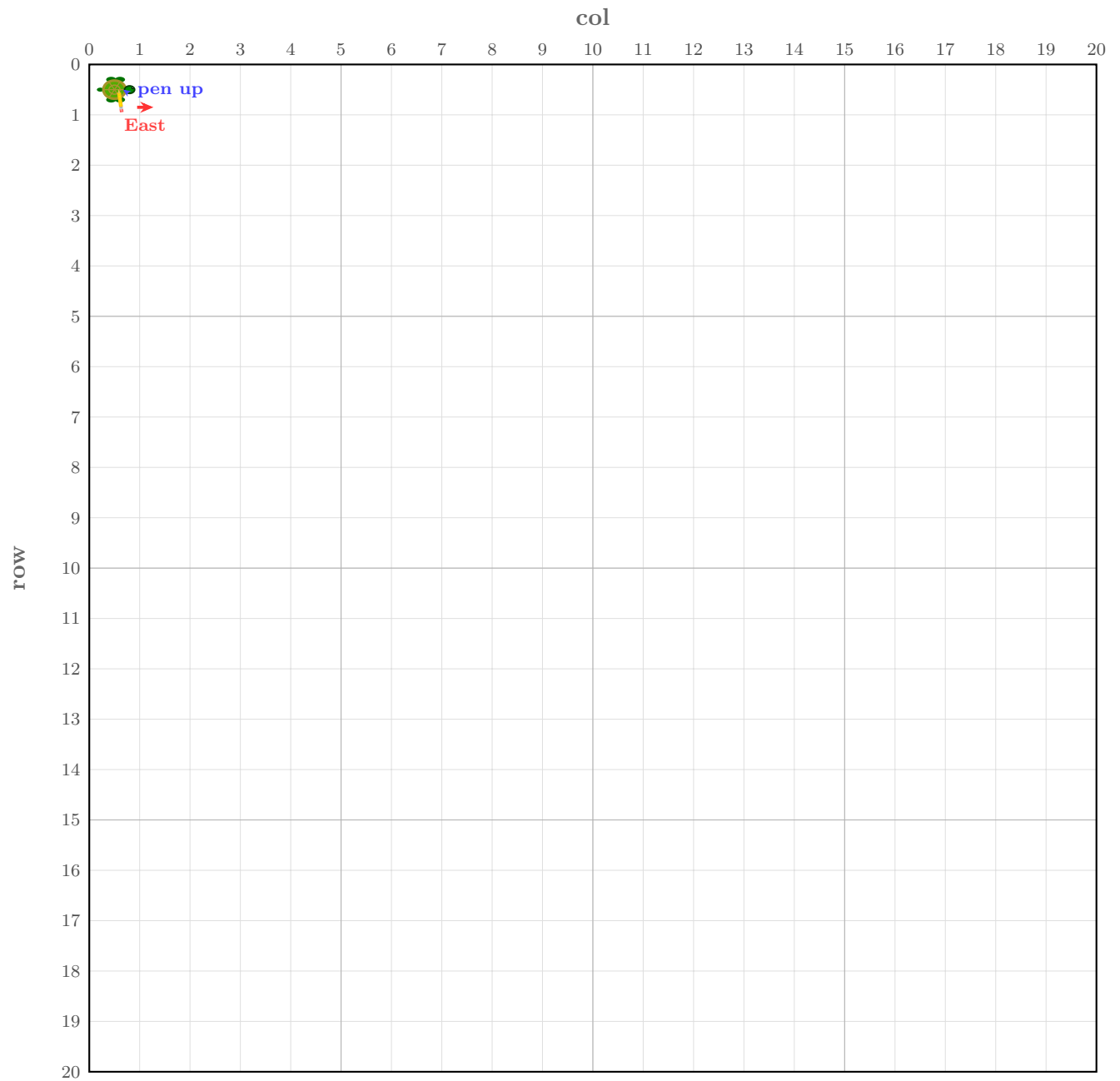
10 // 6 - print floor
11 // 9 - stop
12 run();
13
14 // Runs the turtle graphics program using a predefined list of commands.
15 // Each inner vector represents one command:
16 // - commands with no parameter: {command} e.g. {1}, {2}, {3}
17 // - commands with one parameter: {command, steps} e.g. {5, 3}
18 // Calls restart() before processing commands.
19 // @param commands: a 2D vector of integer commands to execute
20 run(const std::vector<std::vector<int>>& commands);

```

6.4.1 Version 1: Reading Commands from the Console

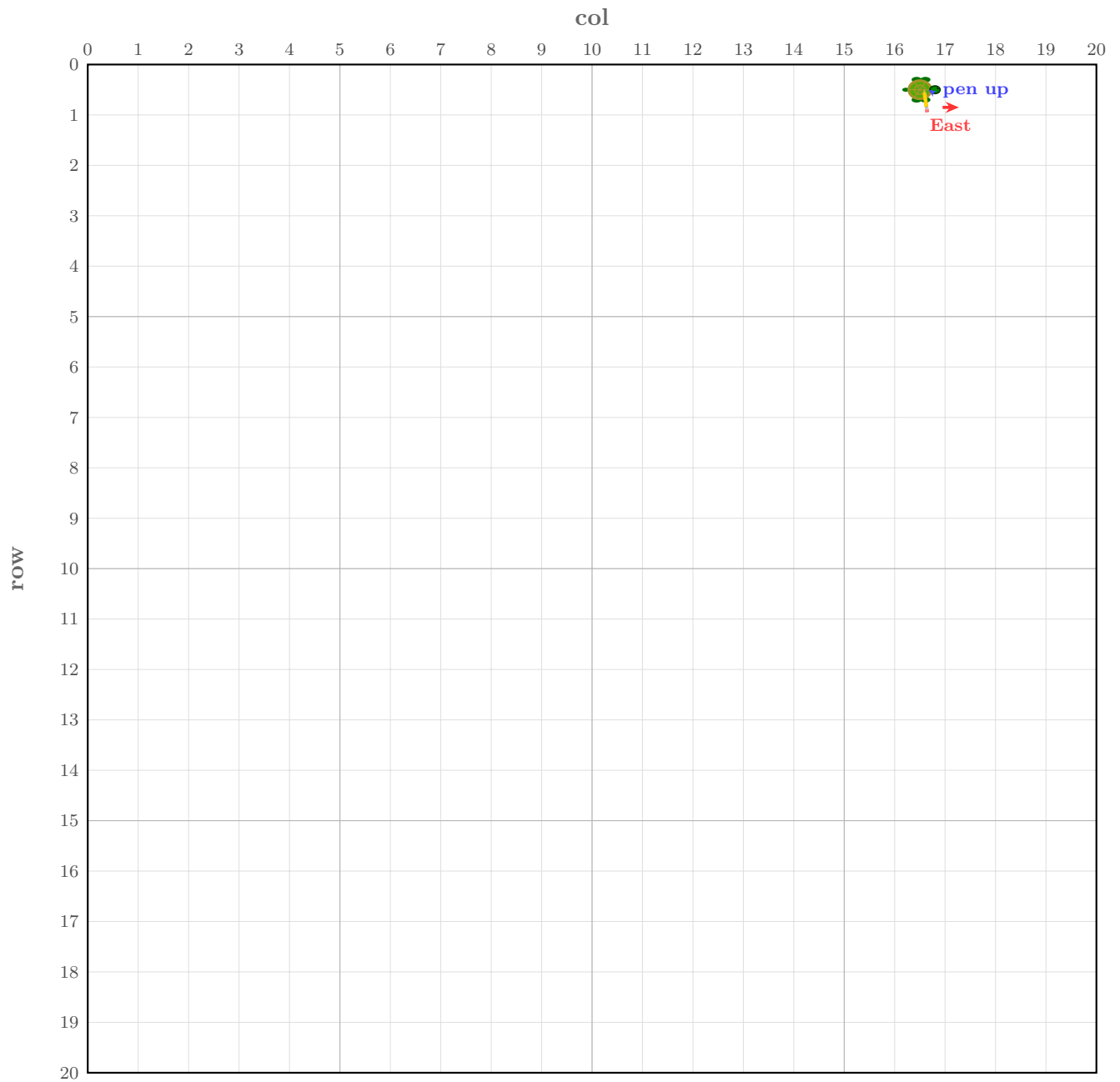
Suppose we do the following.

1. Let a turtle start from (0, 0) of a 20 x 20 grid, pen up, facing east.



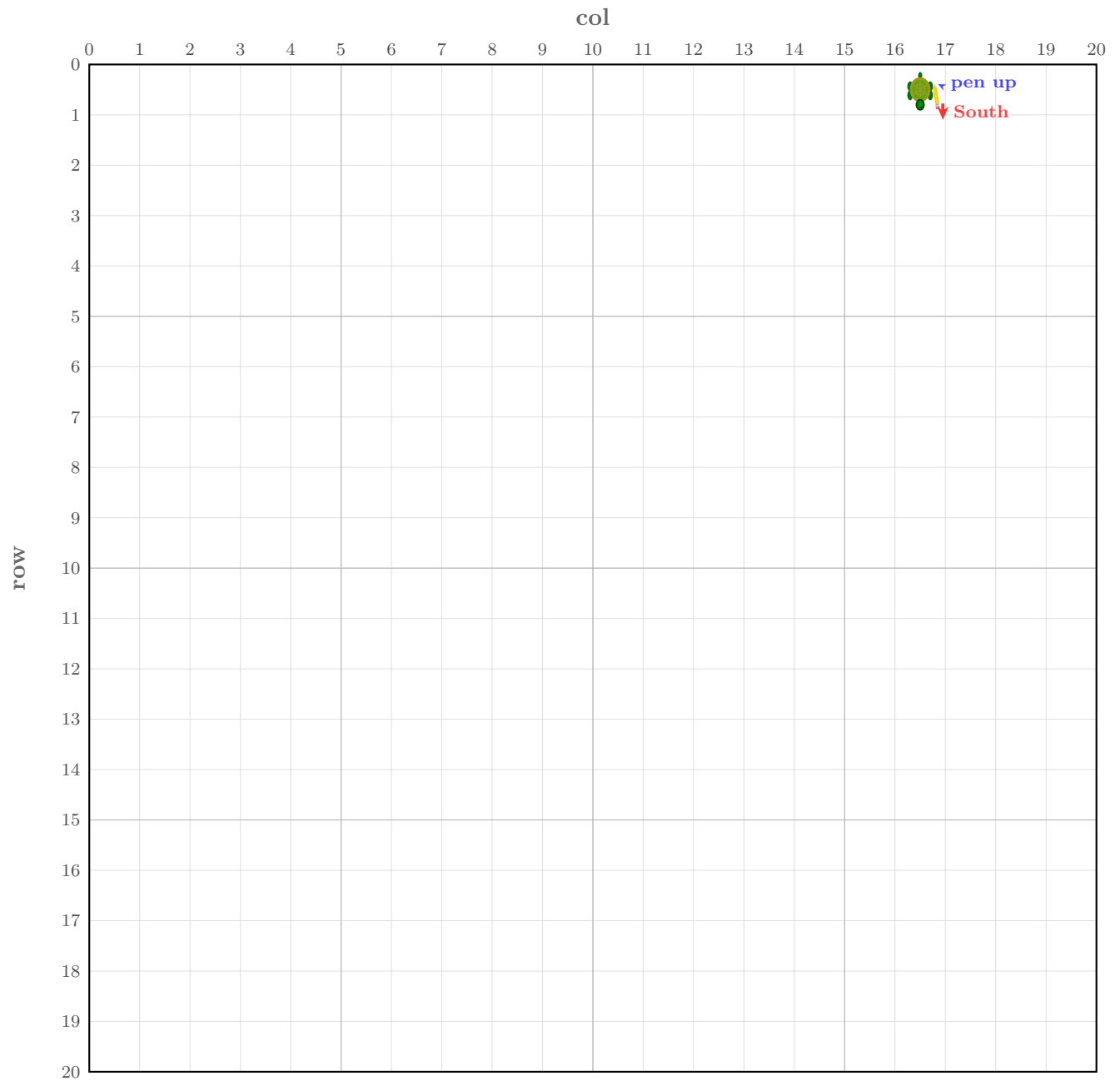
Turtle Graphics — 20×20 grid Start: (col = 0, row = 0) Facing: East Pen: UP

2. Enter command 5 (Move). This command requires a parameter for the number of steps; enter 16. The turtle then moves 16 steps east from its current position, ending at row and column indices (0, 16).



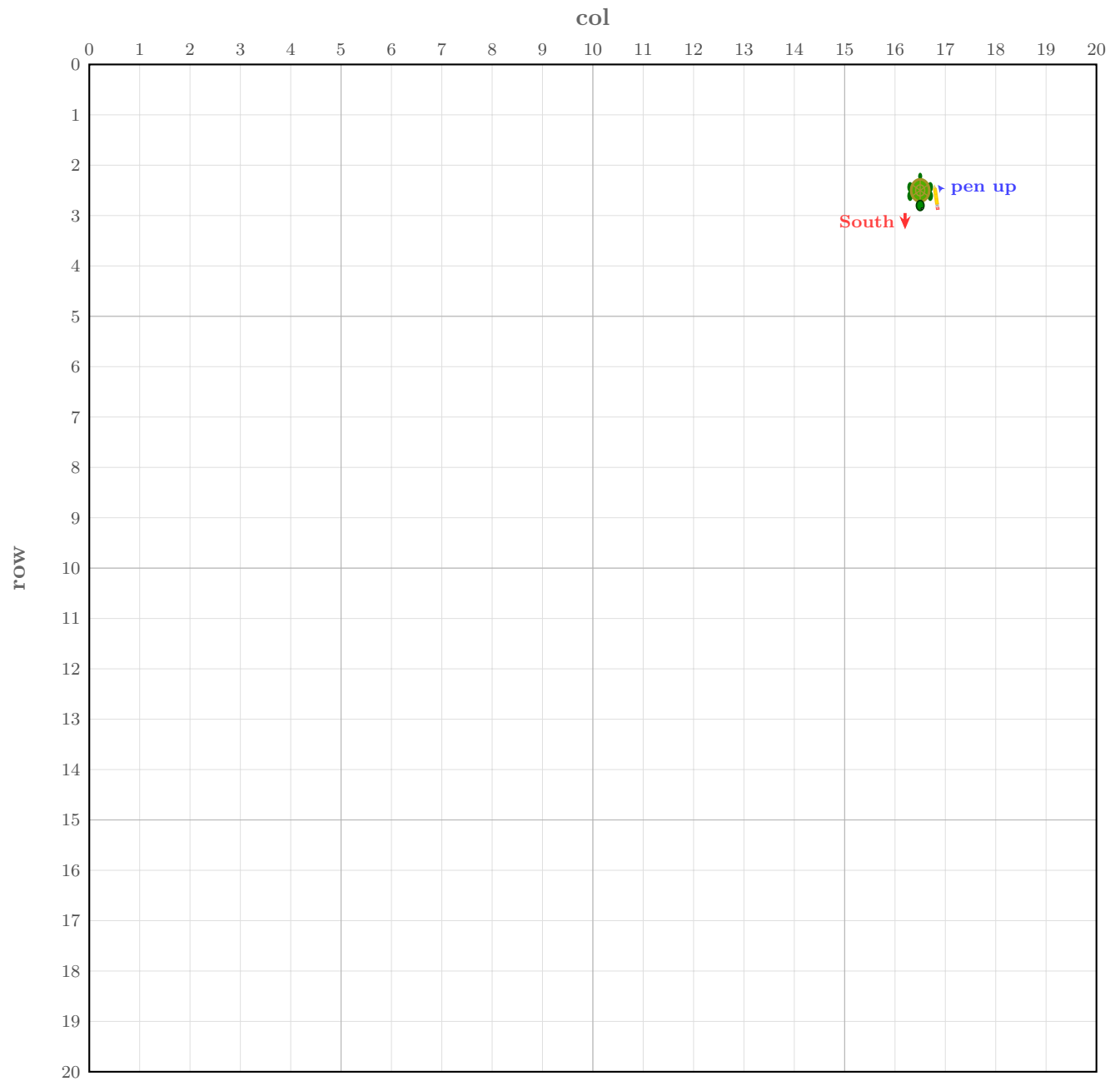
Turtle Graphics — 20×20 grid Start: (col = 16, row = 0) Facing: East Pen: UP

3. Enter command 3 (Turn Right). The turtle rotates 90 degrees clockwise, changing its heading from east to south.



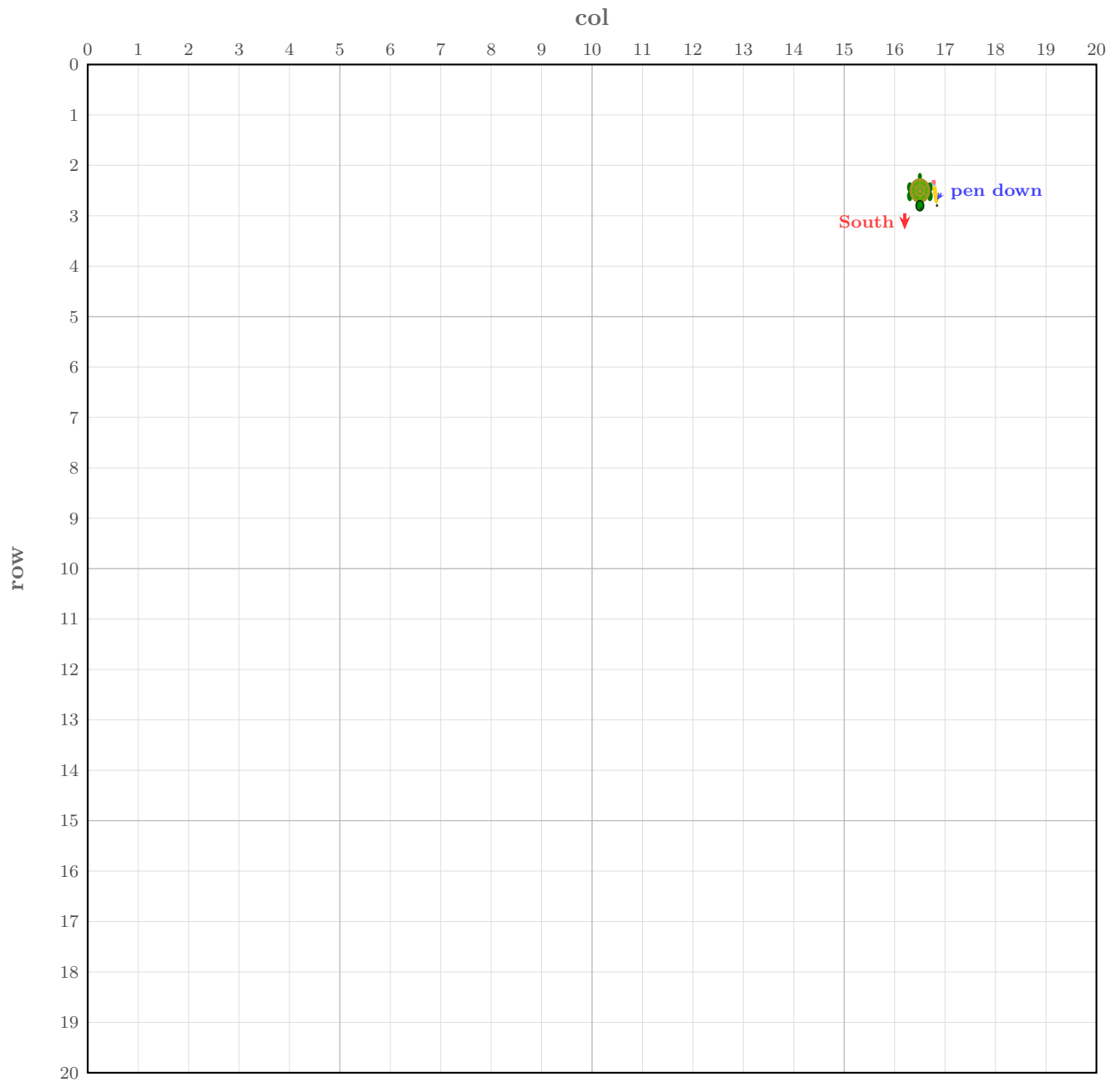
Turtle Graphics — 20×20 grid Pos: (col = 16, row = 0) Facing: South Pen: UP

4. Enter command 5 (Move). This command requires an additional parameter for the number of steps; enter 2. The turtle then moves 2 steps in its current direction.



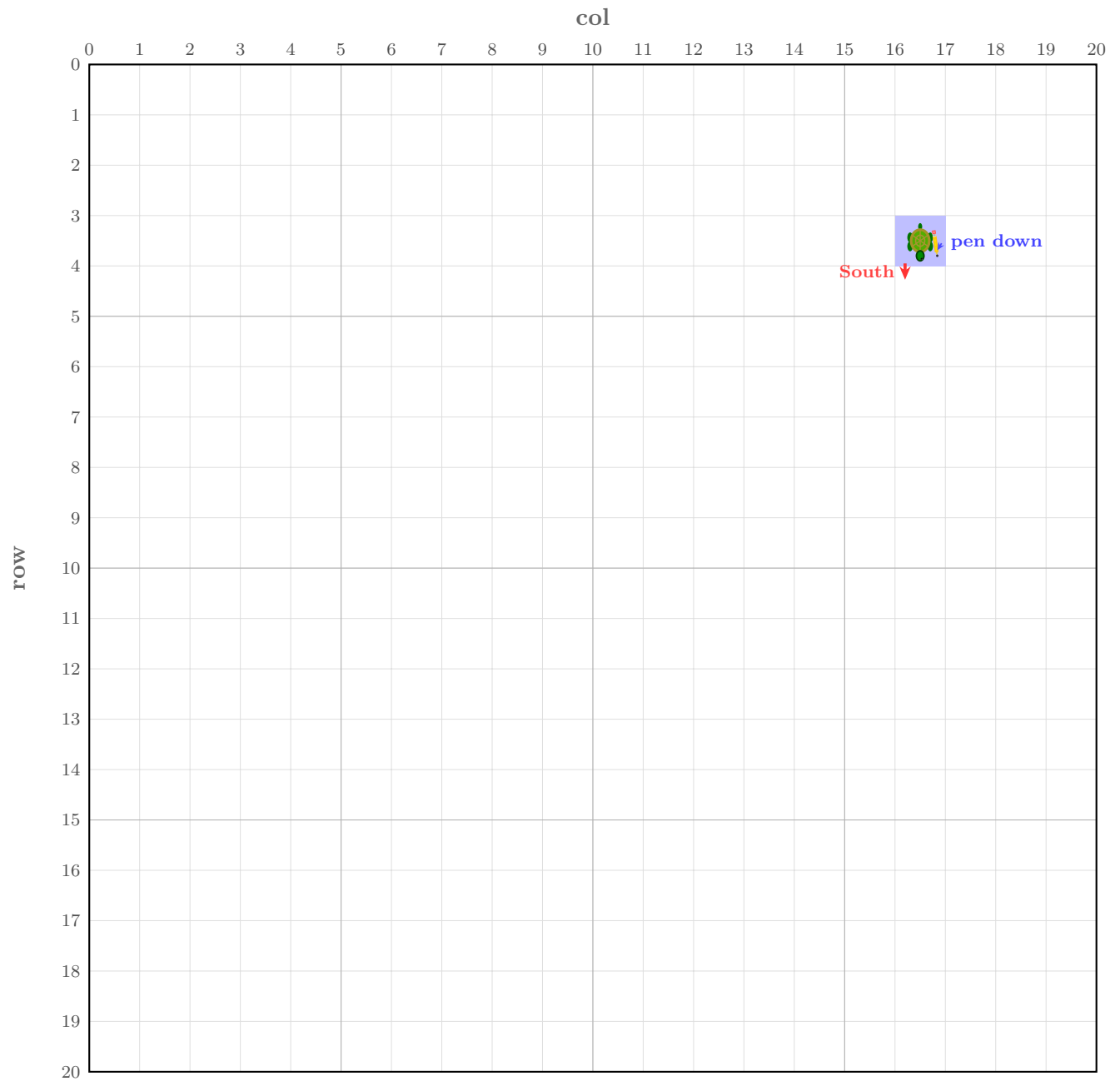
Turtle Graphics — 20×20 grid Pos: (col = 16, row = 2) Facing: South Pen: UP

5. Enter command 2 (Pen Down). The turtle lowers the pen to the drawing surface.



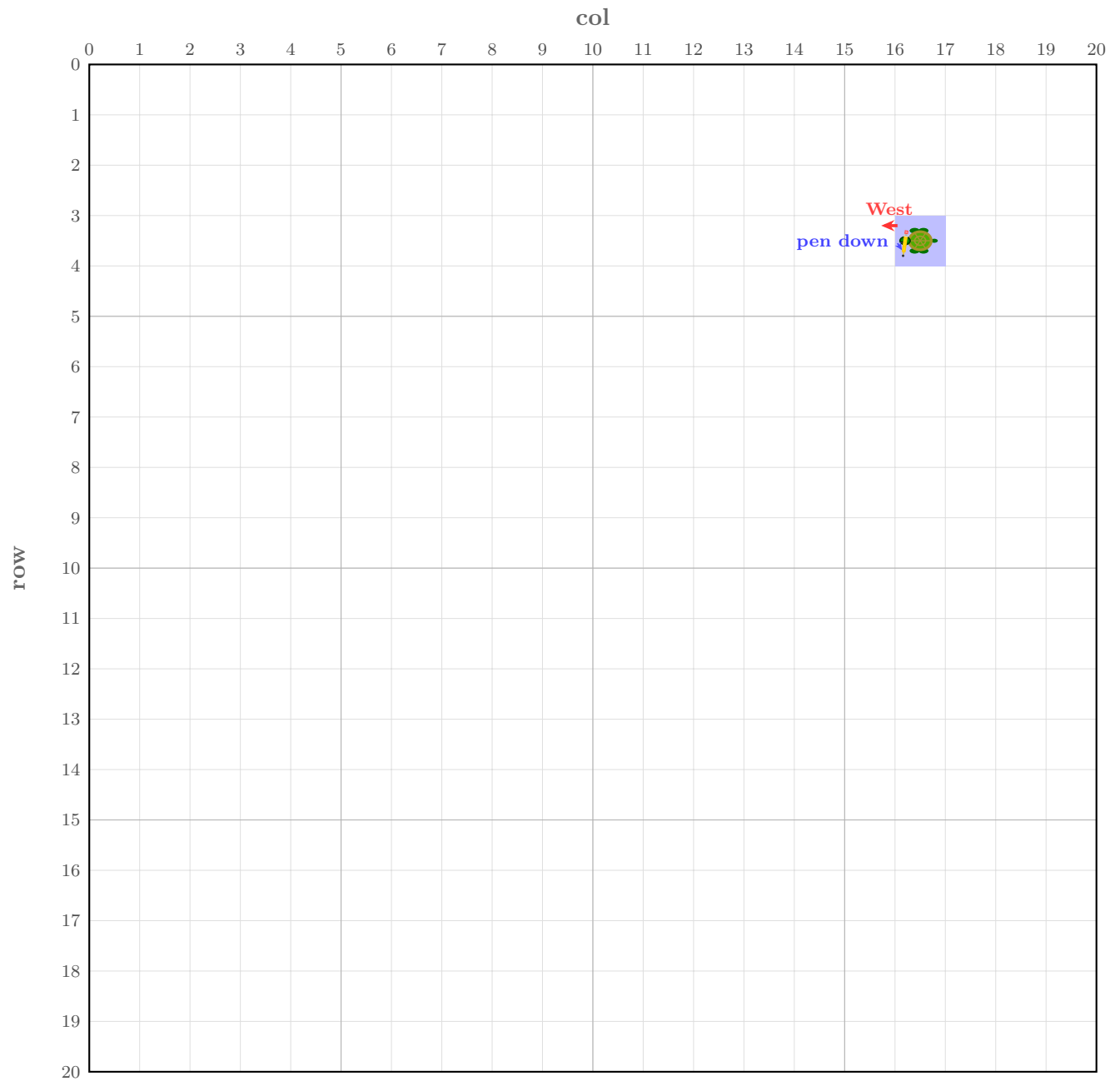
Turtle Graphics — 20×20 grid Pos: (col = 16, row = 2) Facing: South Pen: DOWN

6. Execute command 5 (Move) with a parameter of 1. The turtle moves one step south. While the pen is down, it would typically leave a mark (such as an asterisk); however, to avoid being obscured by the turtle icon, the cell color changes to blue as a compromise.



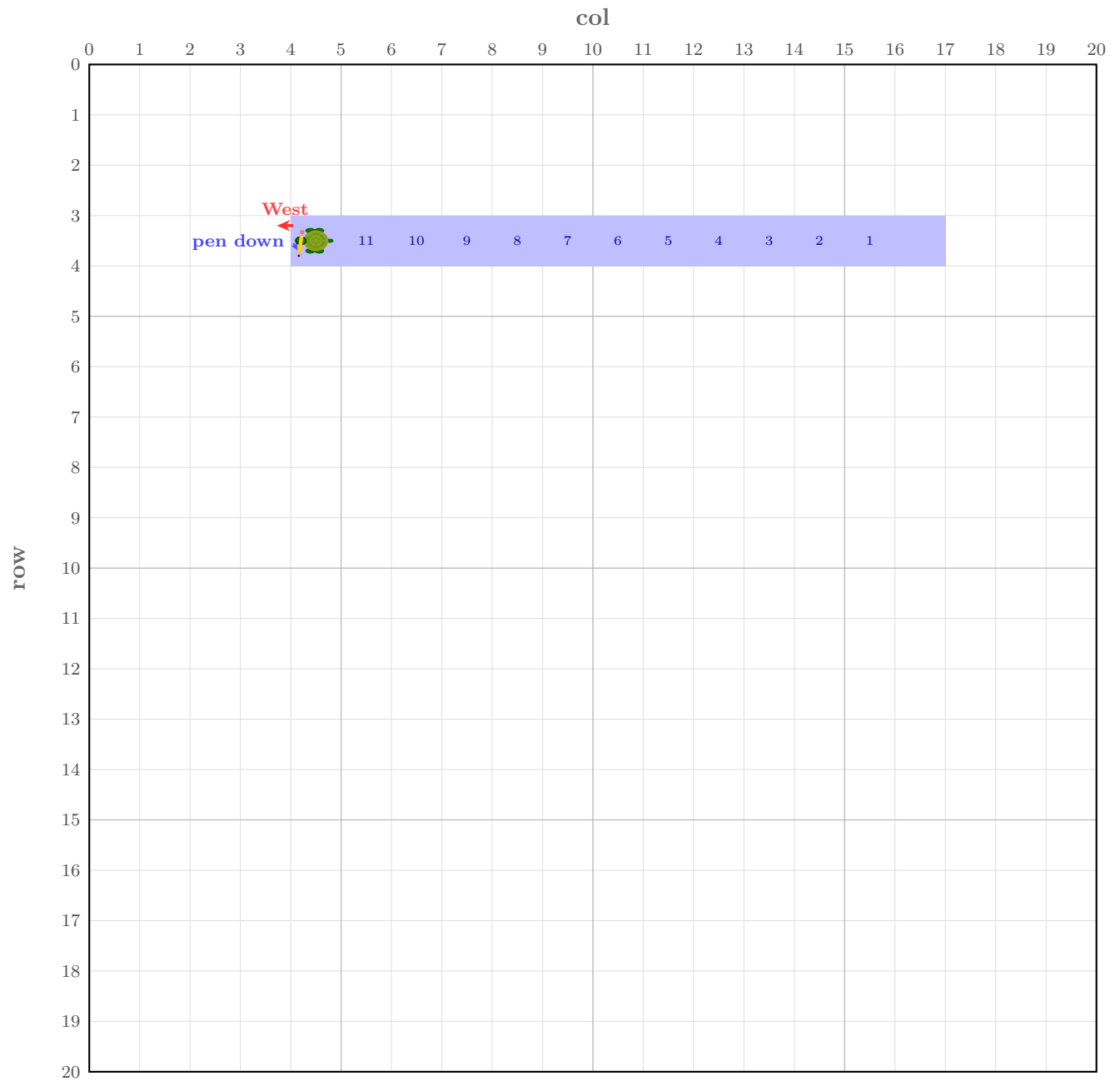
Turtle Graphics — 20×20 grid Pos: (col = 16, row = 3) Facing: South Pen: DOWN

7. Execute command 3 (Turn Right). The turtle rotates clockwise, changing its heading from south to west.



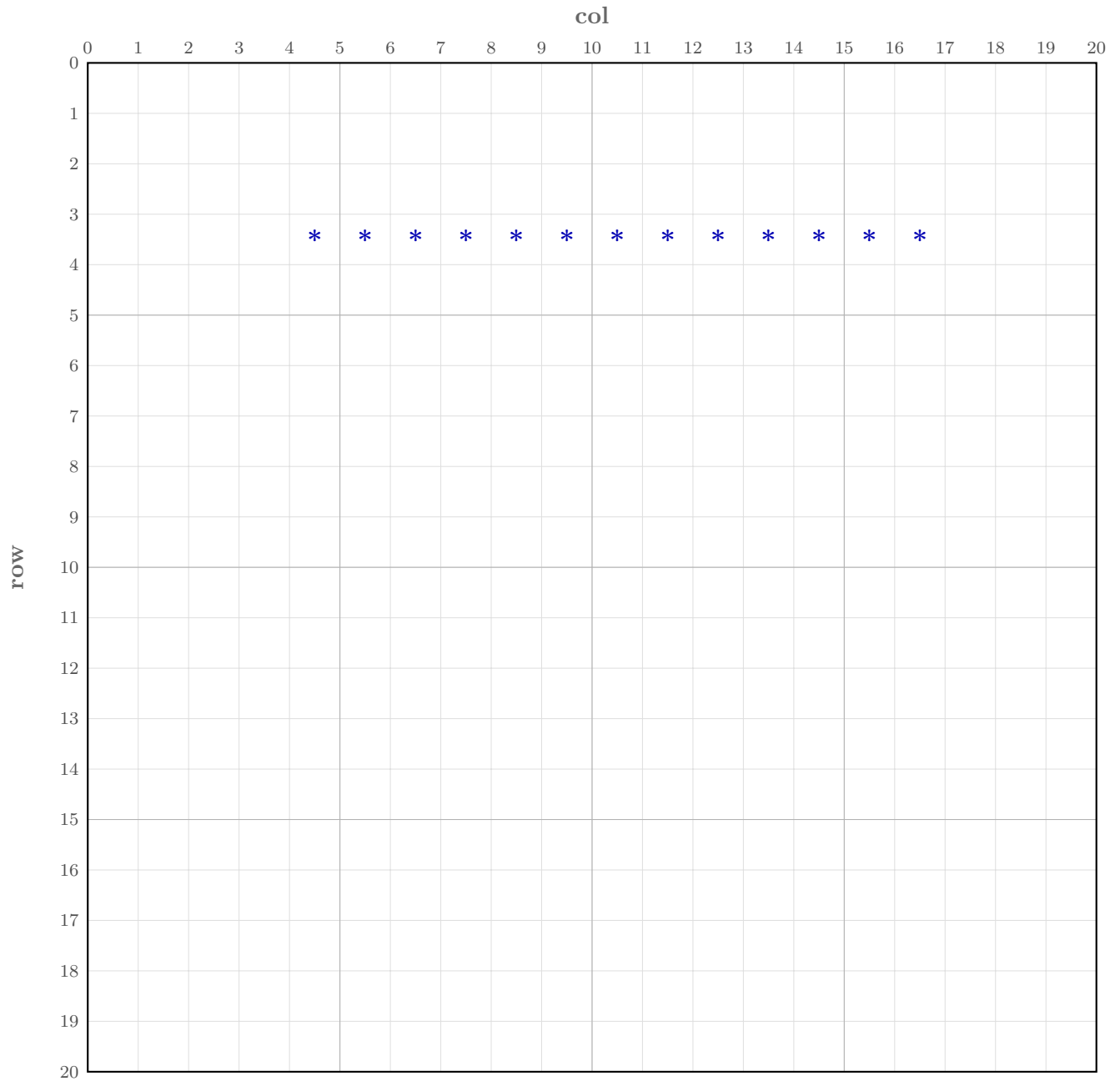
Turtle Graphics — 20×20 grid Pos: (col = 16, row = 3) Facing: West Pen: DOWN

8. Execute command 5 (Move) with a parameter of 12. The turtle moves 12 steps in the current direction, changing the color of each passing cell to blue.



Turtle Graphics — 20×20 grid Pos: (row = 3, col = 4) Facing: West Pen: DOWN

9. Execute command 6 (Print). Display the current state of the floor grid.



Floor status after Command 6 (pen up) Painted: row = 3, col = 4 – 16

10. Execute command 9 to finish the game.

6.4.2 Version 2: Reading Commands from a Two-Dimensional Vector

The commands from Section 6.4.1 can be organized into a two-dimensional vector as follows. Each element of `commands` is a one-dimensional vector containing one or two integers enclosed in braces. Reason: command 5 (Move) needs an additional parameter to represent the number of steps to move.

```
1  std::vector<std::vector<int>>> commands =
2  { {5,16}, {3}, {5,2}, {2}, {5,1}, {3}, {5,12}, {6}, {9} };
```

6.4.3 Comparison of Run Method Versions 1 and 2

Both versions of the `run()` method must call the `restart()` method of the `TurtleGraphics` class. This clears the floor, places the turtle in the top-left corner, sets the pen to the "up" position, and ensures the turtle is facing east.

Logic	Version 1: Console	Version 2: 2D Vector
Execution	Read and process commands from the console sequentially until command 9 is entered.	Iterate through the 2D vector. For each inner vector, extract the first element as the command and process it.
Command Processing	If the command is 5, read one additional parameter from the console to move the turtle. For all other commands, execute the corresponding Turtle or Floor methods.	If the command is 5, use the second element of the inner vector as the movement parameter. For all other commands, execute the corresponding Turtle or Floor methods.

6.4.4 Hint for Method `processCommand`

The private method `processCommand` is called internally by both `run()` and `run(commands)` to reduce code redundancy. Notice that it has a parameter `inputFromConsole`:

- When called from `run()`, command 5 reads the number of steps from `std::cin`.
- When called from `run(commands)`, command 5 uses the `numSteps` parameter directly.

Think about what value of `inputFromConsole` corresponds to each case, and how `numSteps` should be set in each call.

6.5 Download `input3.txt`

These are the commands to draw digit 3 in a 20 x 20 grid.

Download link: [input3.txt](#)

6.6 Test `TurtleGraphics.cpp`

Download link: [TurtleGraphicsUnitTest.cpp](#)

Compile and link all source files using the following command.

```
g++ -o tg Floor.cpp Turtle.cpp TurtleGraphics.cpp TurtleGraphicsUnitTest.cpp
```

Run the resulting executable to verify your implementation.

Test default constructor of `TurtleGraphics` class using

```
./tg A
```

Test restart method using

```
./tg B
```

Test run commands stored in a two-dimensional vector to draw digit 5 using

```
./tg C
```

Test run commands with console input to draw digit 3 using

```
./tg D < input3.txt
```

Or you can test the commands by inputting one a time from the console using

```
./tg D
```

6.7 Submission for Task C

Submit `TurtleGraphics.cpp` to gradescope.

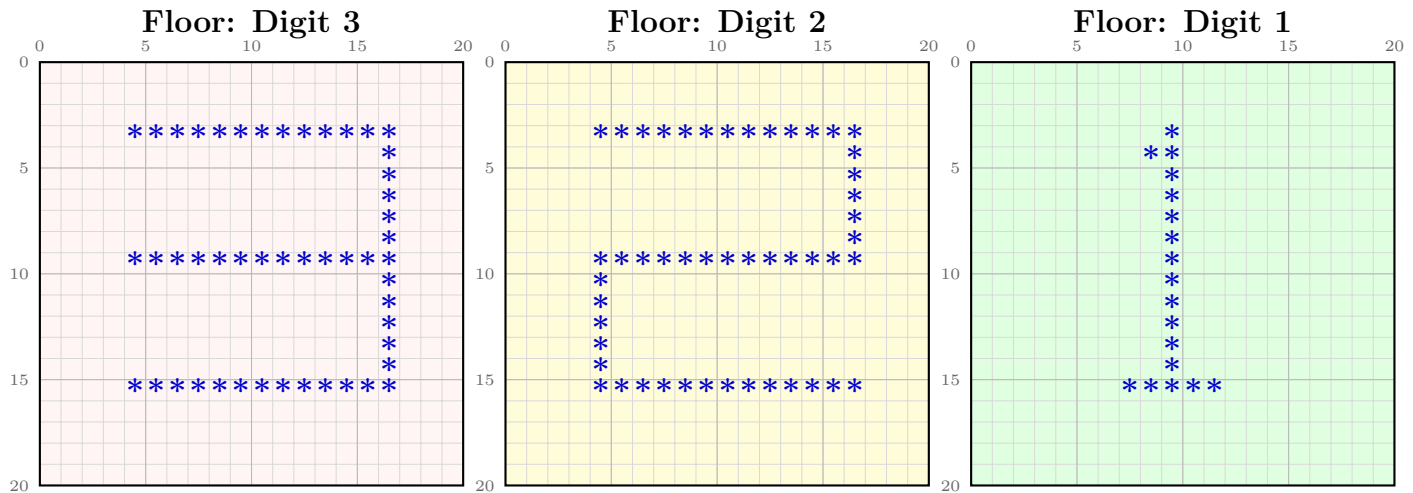
7 Task D: Draw Shapes Side by Side

Now we have defined `Floor`, `Turtle`, and `TurtleGraphics` classes, we would like to draw shapes side by side.

7.1 Drawing Digits of an Integer Side-by-Side

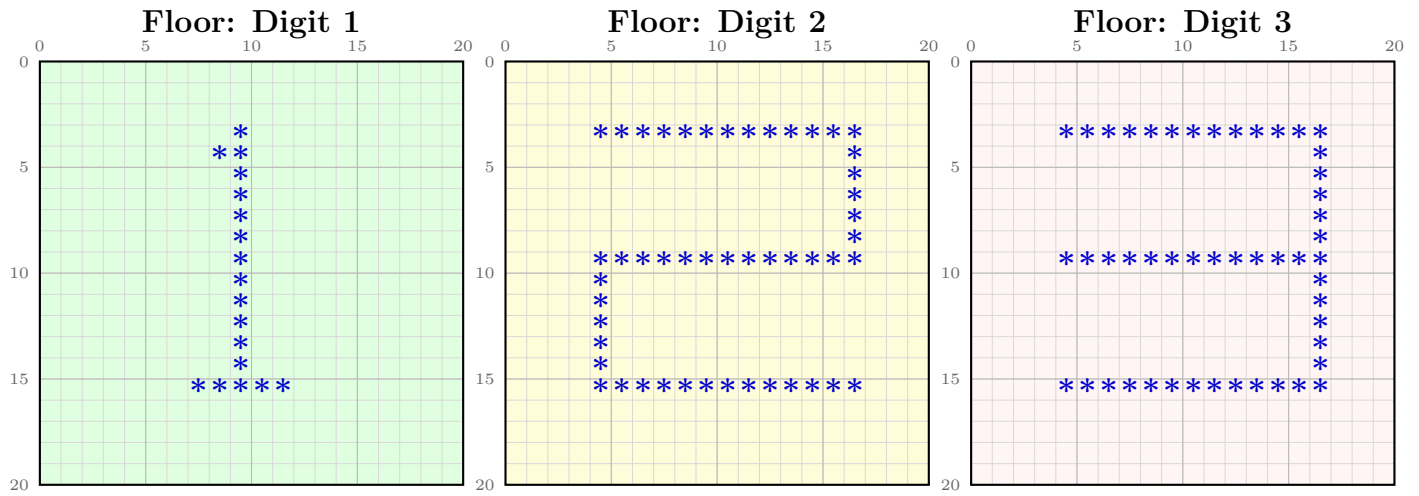
Given an integer, perform the following steps:

1. Extract the digits from right to left (from least significant to most significant).
2. For each digit, call the `run()` method of the `TurtleGraphics` class to draw the corresponding digit. After drawing, retrieve the floor object and add it to the end of a vector.
3. For example, if the input integer is 123, the digit 3 is extracted and stored first, followed by 2, and finally 1. The resulting vector maintains this order, as illustrated below:



Stack order: digit 3 pushed first (leftmost floor) — digit 1 pushed last (rightmost floor)

However, we would like to show the original integer 123 as follows.



Floors left to right: digit 1, digit 2, digit 3

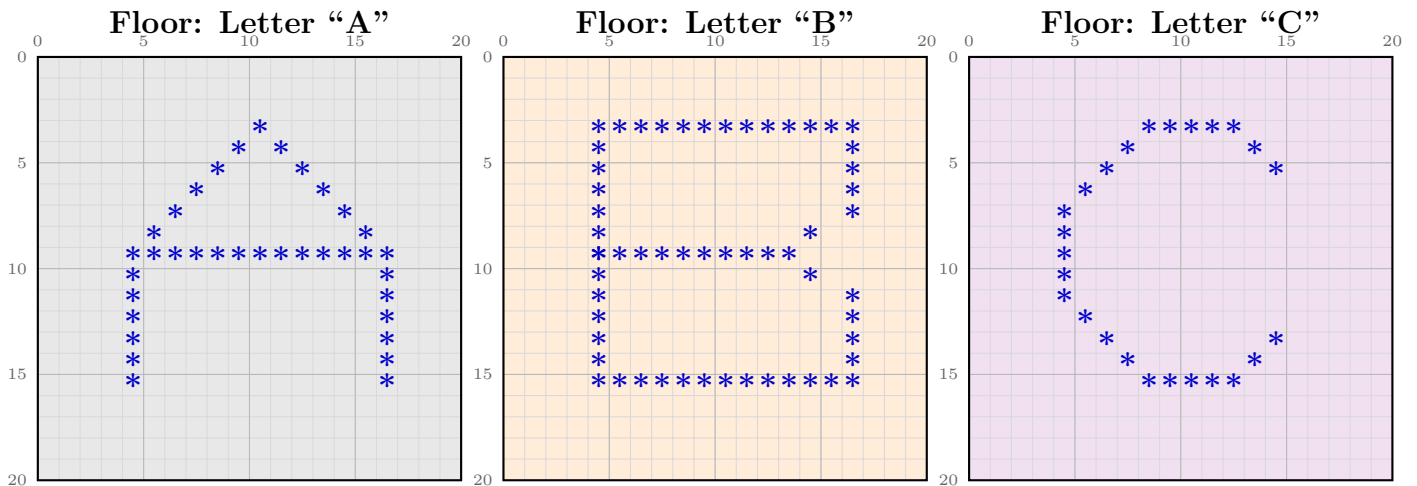
The rendering logic is outlined in the pseudocode below:

```

1 Outer loop: Iterate through each row of the floor matrix.
2
3 Middle loop: Iterate through the shapes in REVERSE order (from the last index to
  the first).
4 // Reason: Digits were stored least-significant-first.
5 // Reversing the order prints them from left to right (most significant digit first
  ).
6
7 Inner loop: Iterate through each column of the current shape's row.
```

For more details, see the void `display(int num)` method in the code snippet in Section 7.3.

7.2 Drawing Characters of a String Side-by-Side



Push order: A first (leftmost floor), B second, C last (rightmost floor)

When processing the characters of a string, the shapes are pushed into a vector from left to right, matching the final printing sequence. The rendering logic is outlined in the pseudocode below:

```
1 Outer loop: Iterate through each row of the floor matrix.
2
3 Middle loop: Iterate through the shapes in forward order (from the first index to the
  last).
4
5 Inner loop: Iterate through each column of the current shape's row.
```

For more details, see the `void display(const std::string& str)` method in the code snippet in Section 7.3.

7.3 Skeleton Code for `drawSideBySide.cpp`

Here is the skeleton of `drawSideBySide.cpp`, downloaded from [drawSideBySide.cpp](#). Note that this file starts with a small letter, since it is not a file to define a class.

```
1 //File: drawSideBySide.cpp
2 #include <iostream>
3 #include <string>
4 #include <vector>
5 #include <cctype> //isalpha, isdigit, isspace
6 #include <climits> //INT_MAX
7 #include "TurtleGraphics.hpp"
8
9 std::vector<std::vector<int>> CMDS_CAP_LETTERS[] = {
10     //std::vector<std::vector<int>> commandsA =
11     {
12         // tip at (3,10): approach from (2,10) going south
13         {1}, //pen up
14         {3}, //turn right -> SOUTH (start facing EAST, right=SOUTH)
15         {5,2}, //move south -> (2,0)... wait need to go east first
16         {4}, //turn left -> EAST
17         {5,10}, //move east -> (2,10)
18         {3}, //turn right -> SOUTH
19         {2}, //pen down
20         {5,1}, //draw * at (3,10)
21
22         // left diagonal: (4,9), (5,8), (6,7), (7,6), (8,5)
23         {1}, //pen up
24         {3}, //turn right -> WEST
25         {5,1}, //move west -> (3,9)
26         {4}, //turn left -> SOUTH
27         {2}, //pen down
28         {5,1}, //draw * at (4,9)
29
30         {1}, //pen up
```



```

31 {3}, //turn right -> WEST
32 {5,1}, //move west -> (4,8)
33 {4}, //turn left -> SOUTH
34 {2}, //pen down
35 {5,1}, //draw * at (5,8)
36
37 {1}, //pen up
38 {3}, //turn right -> WEST
39 {5,1}, //move west -> (5,7)
40 {4}, //turn left -> SOUTH
41 {2}, //pen down
42 {5,1}, //draw * at (6,7)
43
44 {1}, //pen up
45 {3}, //turn right -> WEST
46 {5,1}, //move west -> (6,6)
47 {4}, //turn left -> SOUTH
48 {2}, //pen down
49 {5,1}, //draw * at (7,6)
50
51 {1}, //pen up
52 {3}, //turn right -> WEST
53 {5,1}, //move west -> (7,5)
54 {4}, //turn left -> SOUTH
55 {2}, //pen down
56 {5,1}, //draw * at (8,5)
57
58 // right diagonal: (4,11),(5,12),(6,13),(7,14),(8,15)
59 {1}, //pen up
60 {4}, //turn left -> EAST (from SOUTH)
61 {4}, //turn left -> NORTH
62 {5,5}, //move north -> (3,5)
63 {3}, //turn right -> EAST
64 {5,6}, //move east -> (3,11)
65 {3}, //turn right -> SOUTH
66 {2}, //pen down
67 {5,1}, //draw * at (4,11)
68
69 {1}, //pen up
70 {4}, //turn left -> EAST
71 {5,1}, //move east -> (4,12)
72 {3}, //turn right -> SOUTH
73 {2}, //pen down
74 {5,1}, //draw * at (5,12)
75
76 {1}, //pen up

```

```

77 {4}, //turn left -> EAST
78 {5,1}, //move east -> (5,13)
79 {3}, //turn right -> SOUTH
80 {2}, //pen down
81 {5,1}, //draw * at (6,13)
82
83 {1}, //pen up
84 {4}, //turn left -> EAST
85 {5,1}, //move east -> (6,14)
86 {3}, //turn right -> SOUTH
87 {2}, //pen down
88 {5,1}, //draw * at (7,14)
89
90 {1}, //pen up
91 {4}, //turn left -> EAST
92 {5,1}, //move east -> (7,15)
93 {3}, //turn right -> SOUTH
94 {2}, //pen down
95 {5,1}, //draw * at (8,15)
96
97 // crossbar row 9, cols 4-16
98 {1}, //pen up
99 {5,1}, //move south -> (8,15) ... need to navigate to (9,3)
100 {3}, //turn right -> WEST
101 {5,12}, //move west -> (8,3)
102 {4}, //turn left -> SOUTH
103 {4}, //turn left -> EAST
104 {2}, //pen down
105 {5,13}, //draw crossbar cols 4-16
106
107 // left leg col 4, rows 10-15
108 {1}, //pen up
109 {4}, //turn left -> NORTH (from EAST)
110 {4}, //turn left -> WEST
111 {5,12}, //move west -> (9,4)
112 {3}, //turn right -> NORTH
113 {3}, //turn right -> EAST
114 {3}, //turn right -> SOUTH
115 {2}, //pen down
116 {5,6}, //draw left leg rows 10-15
117
118 // right leg col 16, rows 10-15
119 {1}, //pen up
120 {4}, //turn left -> EAST
121 {4}, //turn left -> NORTH
122 {5,6}, //move north -> (9,4)... need (9,16)

```

```

123 {3}, //turn right -> EAST
124 {5,12}, //move east -> (9,16)
125 {3}, //turn right -> SOUTH
126 {2}, //pen down
127 {5,6}, //draw right leg rows 10-15
128
129 {1}, //pen up
130 {6}, //print
131 },
132
133 //TurtleGraphics tgA;
134 //tgA.run(commandsA);
135
136 //std::vector<std::vector<int>> commandsB =
137 { // pen up, move to (2,4), draw left vertical col4 rows 3-15
138 {1}, //pen up
139 {5,4}, //move east 4 steps
140 {3}, //turn right, from east to south
141 {5,2}, //move south 2 steps -> (2,4)
142 {2}, //pen down
143 {5,13}, //draw left vertical from (3,4) to (15,4)
144
145 // bottom horizontal row15 cols 5-16
146 {4}, //turn left, from south to east
147 {5,12}, //draw bottom horizontal cols 5-16
148
149 // bottom right col16 rows 14-11
150 {1}, //pen up
151 {4}, //turn left, from east to north
152 {2}, //pen down
153 {5,4}, //draw col16 rows 14 down to 11
154
155 // bottom curve at (10,14)
156 {1}, //pen up
157 {5,1}, //move north 1 -> (10,16)
158 {4}, //turn left, from north to west
159 {5,2}, //move west 2 -> (10,14)
160 {3}, //turn right, from west to north
161 {5,1}, //move north 1 -> (9,14)
162 {3}, //turn right, from north to east
163 {3}, //turn right, from east to south
164 {2}, //pen down
165 {5,1}, //draw bottom curve at (10,14)
166
167 // middle horizontal row9 cols 4-13
168 {1}, //pen up

```

```

169 {3}, //turn right, from south to west
170 {3}, //turn right, from west to north
171 {5,1}, //move north 1 -> (9,14)
172 {3}, //turn right, from north to east
173 {3}, //turn right, from east to south
174 {3}, //turn right, from south to west
175 {5,11}, //move west 11 -> (9,3)
176 {3}, //turn right, from west to north
177 {3}, //turn right, from north to east
178 {2}, //pen down
179 {5,10}, //draw middle horizontal cols 4-13
180
181 // top curve at (8,14)
182 {1}, //pen up
183 {3}, //turn right, from east to south
184 {3}, //turn right, from south to west
185 {3}, //turn right, from west to north
186 {5,1}, //move north 1 -> (8,13)
187 {3}, //turn right, from north to east
188 {2}, //pen down
189 {5,1}, //draw top curve at (8,14)
190
191 // top right col16 rows 3-7
192 {1}, //pen up
193 {5,2}, //move east 2 -> (8,16)
194 {4}, //turn left, from east to north
195 {2}, //pen down
196 {5,5}, //draw col16 rows 7 down to 3
197
198 // top horizontal row3 cols 5-15
199 {1}, //pen up
200 {3}, //turn right, from north to east
201 {3}, //turn right, from east to south
202 {3}, //turn right, from south to west
203 {2}, //pen down
204 {5,11}, //draw top horizontal cols 15 down to 5
205
206 {1}, //pen up
207 {6}, //print
208 },
209 //tgA.run(commandsB);
210
211 //std::vector<std::vector<int>> commandsC =
212 { {1},{3},{5,3},{4},{5,7},{2},{5,5}, // top horiz row3 cols 8-12
213   {1},{5,1},{3},{2},{5,1}, // top-right (4,13)
214   {1},{4},{5,1},{3},{2},{5,1}, // top-right (5,14)

```

```

215 {1},{4},{4},{5,2},{4},{5,7},{4},{2},{5,1}, // top-left (4,7)
216 {1},{3},{5,1},{4},{2},{5,1}, // top-left (5,6)
217 {1},{3},{5,1},{4},{2},{5,1}, // top-left (6,5)
218 {1},{3},{5,1},{4},{2},{5,5}, // left vert col4 rows 7-11
219 {1},{4},{5,1},{3},{2},{5,1}, // bot-left (12,5)
220 {1},{4},{5,1},{3},{2},{5,1}, // bot-left (13,6)
221 {1},{4},{5,1},{3},{2},{5,1}, // bot-left (14,7)
222 {1},{5,1},{4},{2},{5,5}, // bot horiz row15 cols 8-12
223 {1},{4},{5,4},{3},{5,3},{3},{1},{5,1}, // bot-right (12,15), change from
    {2},{5,1} to {1},{5,1}, do not display the top * in the bottom right diagonal
224 {1},{3},{5,1},{4},{2},{5,1}, // bot-right (13,14)
225 {1},{3},{5,1},{4},{2},{5,1}, // bot-right (14,13)
226 {1},{6}
227 },
228 //tgA.run(commandsC);
229
230 //std::vector<std::vector<int>> commandsD =
231 { {1},{3},{5,2},{4},{5,4},{3},{2},{5,13}, // left vert col4 rows 3-15
232   {1},{4},{4},{5,12},{3},{2},{5,6}, // top horiz row3 cols 5-10
233   {1},{5,2},{3},{2},{5,1}, // top curve (4,12)
234   {1},{4},{5,2},{3},{2},{5,1}, // top curve (5,14)
235   {1},{4},{5,2},{3},{2},{5,6}, // right vert col16 rows 6-11
236   {1},{2},{5,1}, // bot curve (12,16)
237   {1},{3},{5,2},{4},{2},{5,1}, // bot curve (13,14)
238   {1},{3},{5,2},{4},{2},{5,1}, // bot curve (14,12)
239   {1},{5,1},{3},{5,8},{4},{4},{2},{5,6}, // bot horiz row15 cols 5-10
240   {1},{6}
241 },
242 //tgA.run(commandsD);
243
244 //std::vector<std::vector<int>> commandsE =
245 { {1},{3},{5,2},{4},{5,4},{3},{2},{5,13}, // left vert col4 rows 3-15
246   {1},{4},{4},{5,12},{4},{5,1},{4},{4},{2},{5,13}, // top horiz row3 cols 4-16
247   {1},{3},{5,6},{3},{5,13},{4},{4},{2},{5,11}, // middle horiz row9 cols 4-14
248   {1},{3},{5,6},{3},{5,11},{4},{4},{2},{5,13}, // bot horiz row15 cols 4-16
249   {1},{6}
250 },
251 //tgA.run(commandsE);
252
253 //std::vector<std::vector<int>> commandsF =
254 { {1},{3},{5,2},{4},{5,4},{3},{2},{5,13}, // left vert col4 rows 3-15
255   {1},{4},{4},{5,12},{4},{5,1},{4},{4},{2},{5,13}, // top horiz row3 cols 4-16
256   {1},{3},{5,6},{3},{5,13},{4},{4},{2},{5,11}, // middle horiz row9 cols 4-14
257   {1},{6}
258 },
259 //tgA.run(commandsF);

```

```

260
261 //std::vector<std::vector<int>> commandsG =
262 { // left vertical col4 rows 3-15
263     {1}, //pen up
264     {5,4}, //move east 4 steps
265     {3}, //turn right, from east to south
266     {5,2}, //move south 2 -> (2,4)
267     {2}, //pen down
268     {5,13}, //draw left vertical rows 3-15
269
270     // bottom horizontal row15 cols 5-16
271     {4}, //turn left, from south to east
272     {2}, //pen down
273     {5,12}, //draw bottom horizontal cols 5-16
274
275     // right vertical col16 rows 14-9
276     {1}, //pen up
277     {4}, //turn left, from east to north
278     {2}, //pen down
279     {5,6}, //draw col16 rows 14 down to 9
280
281     // middle arm row9 cols 15-10
282     {4}, //turn left, from north to west
283     {5,6}, //draw middle arm cols 15 down to 10
284
285     // top-right curve col16 rows 5-3
286     {3}, //turn right, from west to north
287     {1}, //pen up
288     {5,3}, //move north 3 -> (6,10)
289     {3}, //turn right, from north to east
290     {5,6}, //move east 6 -> (6,16)
291     {3}, //turn right, from east to south
292     {3}, //turn right, from south to west
293     {3}, //turn right, from west to north
294     {2}, //pen down
295     {5,3}, //draw col16 rows 5,4,3
296
297     // top horizontal row3 cols 15-5
298     {1}, //pen up
299     {3}, //turn right, from north to east
300     {3}, //turn right, from east to south
301     {3}, //turn right, from south to west
302     {2}, //pen down
303     {5,11}, //draw top horizontal cols 15 down to 5
304
305     {1}, //pen up

```

```

306     {6}, //print
307 },
308 //tgA.run(commandsG);
309
310 //std::vector<std::vector<int>> commandsH =
311 { {1},{5,4},{3},{5,2},{2},{5,13},          // left vertical col4 rows 3-15
312   {1},{4},{5,12},{4},{5,13},{3},{3},{2},{5,13}, // right vertical col16 rows 3-15
313   {1},{3},{3},{5,6},{3},{3},{3},{2},{5,12}, // middle horiz row9 cols 5-16
314   {1},{6},
315 },
316 //tgA.run(commandsH);
317
318 //std::vector<std::vector<int>> commandsI =
319 { {1},{3},{5,3},{4},{5,4},{2},{5,11},      // top horiz row3 cols 5-15
320   {1},{4},{5,1},{4},{5,5},{4},{2},{5,13},  // center vert col10 rows 3-15
321   {1},{3},{5,6},{4},{4},{2},{5,11},      // bot horiz row15 cols 5-15
322   {1},{6}
323 },
324 //tgA.run(commandsI);
325
326 //std::vector<std::vector<int>> commandsJ =
327 { // top horiz row3 cols 5-15 (11 cells)
328   {1},{3},{5,3},{4},{5,4},{2},{5,11},
329   // vert col10 rows 3-12
330   {1},{4},{5,1},{4},{5,5},{4},{2},{5,10},
331   // curve (13,9)
332   {1},{3},{5,1},{4},{2},{5,1},
333   // curve (14,8)
334   {1},{3},{5,1},{4},{2},{5,1},
335   // left hook col4 rows 12-15
336   {1},{4},{4},{5,3},{4},{5,4},{4},{2},{5,4},
337   // bot horiz row15 cols 4-7
338   {1},{3},{5,1},{4},{4},{2},{5,4},
339   {1},{6}
340 },
341 //tgA.run(commandsJ);
342
343 //std::vector<std::vector<int>> commandsK =
344 { // left vertical col4 rows 3-15
345   {1},{3},{5,2},{4},{5,4},{3},{2},{5,13},
346   // upper diagonal: (8,6),(7,8),(6,10),(5,12),(4,14) -- step 2 per row
347   {1},{4},{4},{5,8},{3},{5,2},{3},{2},{5,1},
348   {1},{4},{4},{5,2},{3},{5,2},{3},{2},{5,1},
349   {1},{4},{4},{5,2},{3},{5,2},{3},{2},{5,1},
350   {1},{4},{4},{5,2},{3},{5,2},{3},{2},{5,1},
351   {1},{4},{4},{5,2},{3},{5,2},{3},{2},{5,1},

```

```

352 // lower diagonal: (10,6),(11,8),(12,10),(13,12),(14,14) -- step 2 per row
353 {1},{5,5},{3},{5,8},{4},{2},{5,1},
354 {1},{4},{5,2},{3},{2},{5,1},
355 {1},{4},{5,2},{3},{2},{5,1},
356 {1},{4},{5,2},{3},{2},{5,1},
357 {1},{4},{5,2},{3},{2},{5,1},
358 {1},{6}
359 },
360 //tgA.run(commandsK);
361
362 //std::vector<std::vector<int>> commandsL =
363 //letter L
364 { {1}, {3}, {5, 2}, {4}, {5, 4}, {3}, {2}, {5, 13}, {1}, {3}, {5, 1}, {4}, {4}, {2},
    {5, 13}, {1}, {6}
365 },
366 //tgA.run(commandsL);
367
368 //std::vector<std::vector<int>> commandsM =
369 { {{1}, {3}, {5, 2}, {4}, {5, 4}, {3}, {2}, {5, 13}, {1}, {4}, {4}, {5, 13}, {3}, {5,
    12}, {3}, {2}, {5, 13}, {1}, {4}, {4}, {5, 12}, {4}, {5, 11}, {4}, {2}, {5, 1},
    {1}, {4}, {5, 1}, {3}, {2}, {5, 1}, {1}, {4}, {5, 1}, {3}, {2}, {5, 1}, {1}, {4},
    {5, 1}, {3}, {2}, {5, 1}, {1}, {4}, {5, 1}, {3}, {2}, {5, 1}, {1}, {4}, {5, 1},
    {3}, {2}, {5, 1}, {1}, {4}, {4}, {5, 6}, {3}, {5, 5}, {3}, {2}, {5, 1}, {1}, {3},
    {5, 1}, {4}, {2}, {5, 1}, {1}, {3}, {5, 1}, {4}, {2}, {5, 1}, {1}, {3}, {5, 1},
    {4}, {2}, {5, 1}, {1}, {3}, {5, 1}, {4}, {2}, {5, 1}, {1}, {3}, {5, 1}, {4}, {2},
    {5, 1}, {1}, {6}}
370 },
371 //tgA.run(commandsM);
372
373 //std::vector<std::vector<int>> commandsN =
374 { {1}, {3}, {5, 2}, {4}, {5, 4}, {3}, {2}, {5, 13}, {1}, {4}, {4}, {5, 13}, {3}, {5,
    12}, {3}, {2}, {5, 13}, {1}, {4}, {4}, {5, 12}, {4}, {5, 11}, {4}, {2}, {5, 1},
    {1}, {4}, {5, 1}, {3}, {2}, {5, 1}, {1}, {4}, {5, 1}, {3}, {2}, {5, 1}, {1}, {4},
    {5, 1}, {3}, {2}, {5, 1}, {1}, {4}, {5, 1}, {3}, {2}, {5, 1}, {1}, {4}, {5, 1},
    {3}, {2}, {5, 1}, {1}, {4}, {5, 1}, {3}, {2}, {5, 1}, {1}, {4}, {5, 1}, {3}, {2},
    {5, 1}, {1}, {4}, {5, 1}, {3}, {2}, {5, 1}, {1}, {4}, {5, 1}, {3}, {2}, {5, 1},
    {1}, {4}, {5, 1}, {3}, {2}, {5, 1}, {1}, {6}
375 },
376 //tgA.run(commandsN);
377
378
379 //std::vector<std::vector<int>> commandsO =
380 { // top horiz row3 cols 8-12
381   {1},{3},{5,3},{4},{5,7},{2},{5,5},
382   // top-right corner: (4,13),(5,14),(6,15)
383   {1},{5,1},{3},{2},{5,1},

```



```

384 {1},{4},{5,1},{3},{2},{5,1},
385 {1},{4},{5,1},{3},{2},{5,1},
386 // right vert col16 rows 7-11
387 {1},{4},{5,1},{3},{2},{5,5},
388 // bot-right corner: (12,15),(13,14),(14,13)
389 {1},{3},{5,1},{4},{2},{5,1},
390 {1},{3},{5,1},{4},{2},{5,1},
391 {1},{3},{5,1},{4},{2},{5,1},
392 // bot horiz row15 cols 8-12
393 {1},{5,1},{3},{5,6},{4},{4},{2},{5,5},
394 // bot-left corner: (14,7),(13,6),(12,5)
395 {1},{4},{5,2},{4},{5,5},{4},{2},{5,1},
396 {1},{4},{4},{5,2},{4},{5,1},{4},{2},{5,1},
397 {1},{4},{4},{5,2},{4},{5,1},{4},{2},{5,1},
398 // left vert col4 rows 11-7
399 {1},{3},{5,1},{3},{2},{5,5},
400 // top-left corner: (6,5),(5,6),(4,7)
401 {1},{3},{5,1},{4},{2},{5,1},
402 {1},{5,2},{3},{5,1},{3},{2},{5,1},
403 {1},{4},{4},{5,2},{3},{5,1},{3},{2},{5,1},
404 {1},{6}
405 },
406 //tgA.run(commands0);
407
408 //std::vector<std::vector<int>> commandsP =
409 { {1}, {3}, {5, 2}, {4}, {5, 4}, {3}, {2}, {5, 13}, {1}, {4}, {4}, {5, 12}, {4}, {5,
    1}, {4}, {4}, {2}, {5, 13}, {1}, {4}, {5, 1}, {4}, {4}, {2}, {5, 5}, {1}, {5, 1},
    {3}, {5, 1}, {2}, {5, 1}, {1}, {4}, {5, 1}, {3}, {5, 11}, {4}, {4}, {2}, {5, 10},
    {1}, {6}
410 },
411 //tgA.run(commandsP);
412
413 //std::vector<std::vector<int>> commandsQ =
414 { // 3-step round 0 (same as 0)
415 {1},{3},{5,3},{4},{5,7},{2},{5,5},
416 {1},{5,1},{3},{2},{5,1},
417 {1},{4},{5,1},{3},{2},{5,1},
418 {1},{4},{5,1},{3},{2},{5,1},
419 {1},{4},{5,1},{3},{2},{5,5},
420 {1},{3},{5,1},{4},{2},{5,1},
421 {1},{3},{5,1},{4},{2},{5,1},
422 {1},{3},{5,1},{4},{2},{5,1},
423 {1},{5,1},{3},{5,6},{4},{4},{2},{5,5},
424 {1},{4},{5,2},{4},{5,5},{4},{2},{5,1},
425 {1},{4},{4},{5,2},{4},{5,1},{4},{2},{5,1},
426 {1},{4},{4},{5,2},{4},{5,1},{4},{2},{5,1},

```

```

427 {1},{3},{5,1},{3},{2},{5,5},
428 {1},{3},{5,1},{4},{2},{5,1},
429 {1},{5,2},{3},{5,1},{3},{2},{5,1},
430 {1},{4},{4},{5,2},{3},{5,1},{3},{2},{5,1},
431 // diagonal tail (9,10)..(15,16)
432 {1},{5,4},{4},{5,3},{3},{2},{5,1},
433 {1},{4},{5,1},{3},{2},{5,1},
434 {1},{4},{5,1},{3},{2},{5,1},
435 {1},{4},{5,1},{3},{2},{5,1},
436 {1},{4},{5,1},{3},{2},{5,1},
437 {1},{4},{5,1},{3},{2},{5,1},
438 {1},{4},{5,1},{3},{2},{5,1},
439 {1},{6}
440 },
441 //tgA.run(commandsQ);
442
443 //std::vector<std::vector<int>> commandsR =
444 {
445 {1}, {3}, {5, 2}, {4}, {5, 4}, {3}, {2}, {5, 13}, {1}, {4}, {4}, {5, 12}, {4}, {5,
1}, {4}, {4}, {2}, {5, 13}, {1}, {4}, {5, 1}, {4}, {4}, {2}, {5, 5}, {1}, {5,
1}, {3}, {5, 1}, {2}, {5, 1}, {1}, {4}, {5, 1}, {3}, {5, 11}, {4}, {4}, {2}, {5,
10}, {1}, {4}, {4}, {5, 2}, {4}, {2}, {5, 1}, {1}, {4}, {5, 1}, {3}, {2}, {5,
1}, {1}, {4}, {5, 1}, {3}, {2}, {5, 1}, {1}, {4}, {5, 1}, {3}, {2}, {5, 1}, {1},
4}, {5, 1}, {3}, {2}, {5, 1}, {1}, {4}, {5, 1}, {3}, {2}, {5, 1}, {1}, {6}
446 },
447 //tgA.run(commandsR);
448
449 //std::vector<std::vector<int>> commandsS =
450 { // top horiz row3 cols 6-14
451 {1},{3},{5,3},{4},{5,5},{2},{5,9},
452 // top-left 2-step: (4,5),(5,4)
453 {1},{4},{4},{5,9},{4},{2},{5,1},
454 {1},{3},{5,1},{4},{2},{5,1},
455 // left vert col4 row 6 only
456 {1},{2},{5,1},
457 // 5 waist segments (each 2 cells, adjacent)
458 {1},{5,1},{4},{2},{5,2}, // row7: cols 5-6
459 {1},{3},{5,1},{4},{2},{5,2}, // row8: cols 7-8
460 {1},{3},{5,1},{4},{2},{5,2}, // row9: cols 9-10
461 {1},{3},{5,1},{4},{2},{5,2}, // row10: cols 11-12
462 {1},{3},{5,1},{4},{2},{5,3}, // row11: cols 13-15
463 // right vert col16 rows 12-13 only
464 {1},{5,1},{3},{2},{5,2},
465 // bot-right 2-step: (13,16),(14,15)
466 {1},{4},{4},{5,1},{4},{4},{2},{5,1},
467 {1},{3},{5,1},{4},{2},{5,1},

```

```

468 // bot horiz row15 cols 6-14
469 {1},{5,1},{3},{5,10},{4},{4},{2},{5,9},
470 {1},{6}
471 },
472 //tgA.run(commandsS);
473
474 //std::vector<std::vector<int>> commandsT =
475 { {1}, {3}, {5, 3}, {4}, {5, 3}, {2}, {5, 13}, {1}, {4}, {5, 1}, {4}, {5, 6}, {4},
476   {2}, {5, 13}, {1}, {6}
477 },
478 //tgA.run(commandsT);
479
480 //std::vector<std::vector<int>> commandsU =
481 { {1},{3},{5,2},{4},{5,4},{3},{2},{5,10}, // left vert col4 rows 3-12
482   {1},{4},{4},{5,10},{3},{5,12},{3},{2},{5,10}, // right vert col16 rows 3-12
483   {1},{3},{5,11},{4},{2},{5,1}, // bot-left (13,5)
484   {1},{4},{5,1},{3},{2},{5,1}, // bot-left (14,6)
485   {1},{5,1},{4},{2},{5,6}, // bot horiz row15 cols 7-12
486   {1},{4},{5,2},{3},{5,2},{3},{2},{5,1}, // bot-right (14,14)
487   {1},{4},{4},{5,2},{3},{5,1},{3},{2},{5,1}, // bot-right (13,15)
488   {1},{6}
489 },
490 //tgA.run(commandsU);
491
492 //std::vector<std::vector<int>> commandsV =
493 { {1}, {3}, {5, 2}, {4}, {5, 4}, {3}, {2}, {5, 1}, {1}, {4}, {4}, {5, 1}, {3}, {5,
494   12}, {3}, {2}, {5, 1}, {1}, {3}, {5, 12}, {4}, {2}, {5, 1}, {1}, {4}, {4}, {5, 1},
495   {3}, {5, 12}, {3}, {2}, {5, 1}, {1}, {3}, {5, 11}, {4}, {2}, {5, 1}, {1}, {4},
496   {4}, {5, 1}, {3}, {5, 10}, {3}, {2}, {5, 1}, {1}, {3}, {5, 10}, {4}, {2}, {5, 1},
497   {1}, {4}, {4}, {5, 1}, {3}, {5, 10}, {3}, {2}, {5, 1}, {1}, {3}, {5, 9}, {4}, {2},
498   {5, 1}, {1}, {4}, {4}, {5, 1}, {3}, {5, 8}, {3}, {2}, {5, 1}, {1}, {3}, {5, 8},
499   {4}, {2}, {5, 1}, {1}, {4}, {4}, {5, 1}, {3}, {5, 8}, {3}, {2}, {5, 1}, {1}, {3},
500   {5, 7}, {4}, {2}, {5, 1}, {1}, {4}, {4}, {5, 1}, {3}, {5, 6}, {3}, {2}, {5, 1},
501   {1}, {3}, {5, 6}, {4}, {2}, {5, 1}, {1}, {4}, {4}, {5, 1}, {3}, {5, 6}, {3}, {2},
502   {5, 1}, {1}, {3}, {5, 5}, {4}, {2}, {5, 1}, {1}, {4}, {4}, {5, 1}, {3}, {5, 4},
503   {3}, {2}, {5, 1}, {1}, {3}, {5, 4}, {4}, {2}, {5, 1}, {1}, {4}, {4}, {5, 1}, {3},
504   {5, 4}, {3}, {2}, {5, 1}, {1}, {3}, {5, 3}, {4}, {2}, {5, 1}, {1}, {4}, {4}, {5,
505   1}, {3}, {5, 2}, {3}, {2}, {5, 1}, {1}, {3}, {5, 2}, {4}, {2}, {5, 1}, {1}, {4},
506   {4}, {5, 1}, {3}, {5, 2}, {3}, {2}, {5, 1}, {1}, {3}, {5, 1}, {4}, {2}, {5, 1},
507   {1}, {6}
508 },
509 //tgA.run(commandsV);
510
511 //std::vector<std::vector<int>> commandsW =
512 { {1}, {3}, {5, 2}, {4}, {5, 4}, {3}, {2}, {5, 13}, {1}, {4}, {4}, {5, 13}, {3}, {5,
513   12}, {3}, {2}, {5, 13}, {1}, {4}, {4}, {5, 7}, {4}, {5, 6}, {4}, {2}, {5, 1}, {1},

```



```

    1}, {4}, {2}, {5, 1}, {1}, {6}
512 },
513 //tgA.run(commandsZ);
514 };
515
516 std::vector<std::vector<int>> CMDS_DIGITS[] = {
517     //commands0: comments to draw 0
518     { {1}, // pen up, facing east by default
519       {5,4}, // move east 4 steps from (0,1) to (0,4)
520       {3}, // turn right, from east to south
521       {5,3}, // move 3 steps, from (1,4) to (3,4)
522       {4}, // turn left, from south to east again
523
524       {2}, // pen down
525
526       // draw top horizontal line
527       {5,12}, // move 12 steps, from
528
529       // draw right vertical line
530       {3},{5,12},
531
532       // draw bottom horizontal line
533       {3},{5,12},
534
535       // draw left vertical line
536       {3},{5,12},
537
538       // print
539       {6}
540     },
541
542     //commands1: comments to draw 1
543     { {1},{5,3},{3},{5,3},{4},
544       {5,5},
545       {2},{5,1},{3},{5,1},{3},{5,1},
546       {1},{3},{5,1},{3},{5,1},{3},{5,1},
547       {2},{5,11},
548       {4},{5,2},{1},{4},{5,1},{4},{5,2},
549       {2},{4},{5,1},{3},{5,2},
550       {6}
551     },
552
553     //commands2
554     { {1},{5,3},{3},{5,3},{4},
555       {2},{5,13},{3},{5,6},{3},{5,12},{4},{5,6},{4},{5,12},
556       {6}

```

```

557 },
558
559 //commands3
560 { {1},{5,3},{3},{5,3},{4},
561   {2},{5,13},{3},{5,6},
562   {3},{5,12},{1},
563   {4},{5,5},{2},{5,1},{4},
564   {5,12},{4},{5,6},{6}
565 },
566
567 //commands4
568 { {1}, // pen up, facing east by default
569   {5,4}, // move east 4 steps from (0,1) to (0,4), where  $4-1+1 = 4$ 
570   {3}, // turn right, from east to south
571   {5,2}, // move 2 steps, from (1,4) to (2,3), where  $2-1+1 = 2$ 
572
573   {2}, // pen down
574
575   // draw left line
576   {5,7}, // move 12 steps, from (3,4) to (9,4), where  $9-3+1 = 7$ 
577
578   {4}, // turn left, from south to east
579   {5,12}, // move from (9,5) to (9,16), a total of  $16-5+1 = 12$  steps
580
581   // move to one step above the top of vertical line, ie, (2,16), facing south
582   {1}, // pen up
583   {4}, // turn left, from east to north
584   {5,7}, // move from (8,16) to (2,16), a total of  $8-2+1 = 7$  steps
585   {3}, // turn right, from north to east
586   {3}, // turn right, from east to south
587
588   {2}, // pen down
589   {5,13}, // move from (3,16) to (15,16), a total of  $15-3+1 = 13$  steps
590
591   // print
592   {6}
593 },
594
595 //commands5
596 { // facing east in the beginning
597   {1}, // pen up
598   {5,16}, // move from (0,1) to (0,16)
599   {3}, // turn right from east to south
600   {5,2}, // move three steps in the south direction, from (1,16) to (2,16)
601
602   {2}, // pen down

```

```

603 {5,1}, // draw from (3,16) to (3,16)
604 {3}, // trun right, from south to west
605 {5,12}, // draw horizontal line from (3,15) to (3,4)
606
607 // draw left upper vertical line of 5, from (4,4) to (4,9)
608 {4}, // turn left
609 {5,6}, // draw vertical line
610
611 // draw middle horizontal line
612 {4}, // turn left from south to east
613 {5,12}, // from (9,5) to (9,16)
614
615 // draw right lower vertical line
616 {3}, // turn right from east to south
617 {5,6}, // draw 6 steps from (10,16) to (15, 16)
618
619 // draw bottom horizontal line
620 {3}, // turn right from south to west
621 {5,12}, // draw 12 steps from (15,15) to (15,4)
622
623 {6} // print
624 },
625
626 //commands6
627 { // facing east in the beginning
628 {1}, // pen up
629 {5,16}, // move from (0,1) to (0,16)
630 {3}, // turn right from east to south
631 {5,2}, // move three steps in the south direction, from (1,16) to (2,16)
632
633 {2}, // pen down
634 {5,1}, // draw from (3,16) to (3,16)
635 {3}, // trun right, from south to west
636 {5,12}, // draw horizontal line from (3,15) to (3,4)
637
638 // draw left upper vertical line of 5, from (4,4) to (4,15)
639 {4}, // turn left, from west to south
640 {5,12}, // draw vertical line
641
642 // draw bottom horizontal line
643 {4}, // turn left, from south to east
644 {5,12}, // draw 12 steps from (15,5) to (15,16)
645
646 // draw right lower vertical line
647 {4}, // turn left from east to north
648 {5,6}, // draw 6 steps from (16,14) to (16, 9)

```

```

649
650 // draw middle horizontal line
651 {4}, // turn left from north to west
652 {5,12}, // from (9,5) to (9,16)
653
654 // print
655 {6}
656 },
657
658 //commands7
659 { {1}, // pen up
660   {5,3}, // move east 3 steps
661   {3}, // turn right from east, facing south
662   {5,3}, // move south 3 steps
663   {4}, // turn left from south, face east again
664
665   //## draw top horizontal line
666   {2}, // pen down
667   {5,13}, // move 13 steps, from (3,4) to (3,16),
668
669   {3}, // turn right, from east to south
670   {5,12}, // move 12 steps, from (4,16) to (15,16)
671
672   {6} //print out
673 },
674
675 //commands8
676 { {1}, // pen up, facing east by default
677   {5,4}, // move east 4 steps from (0,1) to (0,4)
678   {3}, // turn right, from east to south
679   {5,3}, // move 3 steps, from (1,4) to (3,4)
680   {4}, // turn left, from south to east again
681
682   {2}, // pen down
683
684   // draw top horizontal line
685   {5,12}, // move 12 steps, from
686
687   // draw right vertical line, facing south
688   {3},{5,12},
689
690   // draw bottom horizontal line, facing west
691   {3},{5,12},
692
693   // draw left vertical line, facing north
694   {3},{5,12},

```



```

695
696 // pen up, turn south
697 {1}, // pen up
698 {3}, // turn right, from south to east
699 {3}, // turn right, from east to north
700 {5,6}, // move 6 steps, at (9,4)
701 {4}, // turn left, from north to east
702
703 // draw middle line
704 {2},
705 {5,12},
706
707 // print
708 {6}
709 },
710
711 //commands9
712 { {1}, // pen up, facing east by default
713   {5,4}, // move east 4 steps from (0,1) to (0,4)
714   {3}, // turn right, from east to south
715   {5,3}, // move 3 steps, from (1,4) to (3,4)
716   {4}, // turn left, from south to east again
717
718   {2}, // pen down
719
720   // draw top horizontal line
721   {5,12}, // move 12 steps, from
722
723   // draw right vertical line, facing south
724   {3},{5,12},
725
726   // draw bottom horizontal line, facing west
727   {3},{5,12},
728
729   // draw left top vertical line, facing north
730   {3},
731   {1},
732   {5,5}, // move to (10,4)
733   {2},
734   {5,7}, // move to (3,4)
735
736   // pen up, turn south
737   {1}, // pen up
738   {3}, // turn right, from south to east
739   {3}, // turn right, from east to north
740   {5,6}, // move 6 steps, at (9,4)

```

```

741     {4}, // turn left, from north to east
742
743     // draw middle line
744     {2},
745     {5,12},
746
747     // print
748     {6}
749 },
750 };
751
752 std::vector<std::vector<int>> CMDS_MINUS = {
753     {1}, // pen up
754     {3},{5,9}, // turn right -> SOUTH, move 9 -> (9,0)
755     {4},{5,4}, // turn left -> EAST, move 4 -> (9,4)
756     {2},{5,10}, // pen down, draw east 10 -> row9 cols 5-14
757     {1},{6}
758 };
759
760 void display(int num);
761 void display(const std::string& str);
762
763 int main() {
764     std::cout << "Enter an integer: ";
765     int num;
766     std::cin >> num;
767     std::cout << "integer shape is follows:\n"; //this line is needed
768
769     // TODO: Call display(num).
770
771     // Discard the newline left in the buffer after reading the integer,
772     // so that the subsequent getline call reads correctly.
773     std::cin.ignore(INT_MAX, '\n');
774     std::cout << "Enter a string (only supports capital letters, spaces, and digits: ";
775     std::string str;
776     getline(std::cin, str);
777     std::cout << "string shape is follows:\n"; //this line is needed
778
779     // TODO: Call display(str).
780
781     return 0;
782 }
783
784 void display(int num) {
785     // TODO:
786     // (1) Declare sign as an int initialized to 1.

```

```

787 // (2) If num is negative:
788 //     (a) Set sign to -1.
789 //     (b) Set num to its absolute value (e.g., -123 becomes 123)
790 //         so that num % 10 always returns a non-negative digit.
791
792 // TODO: Declare a TurtleGraphics object named tg.
793
794 // Declare shapes as a vector of Floor objects.
795 std::vector<Floor> shapes;
796 int digit;
797 do {
798     // TODO: Extract the least significant digit of num into variable digit.
799
800     // TODO: Call tg.run() with CMDS_DIGITS[digit] to draw that digit.
801     // Example: if digit is 3, run CMDS_DIGITS[3], which draws the digit 3.
802
803     // TODO: Call tg.getFloor() and append the result to shapes.
804
805     // TODO: Call tg.restart() to clear the floor and reset the turtle
806     //         to position (0, 0) facing east, ready to draw the next digit.
807
808     // TODO: Remove the least significant digit from num.
809     //         Example: if num is 123, it becomes 12.
810
811 } while (...); // TODO: Continue as long as num has remaining digits.
812
813 // TODO: If sign is -1, call tg.run() with CMDS_MINUS to draw the minus sign,
814 //         then append tg.getFloor() to shapes.
815 //
816 // Note: push_back() adds to the end, so digits are stored in reverse order
817 // (least significant first). Example:
818 // display(-123): shapes contains: [3, 2, 1, -]
819 // display(123): shapes contains: [3, 2, 1]
820
821 // Get the first element of shapes to determine row/column dimensions.
822 // shapes is guaranteed non-empty since num has at least one digit.
823 const Floor& floor = ...; // TODO: first element of shapes
824
825 // TODO: Outer loop: iterate over each row of the floor.
826 for (int row = 0; row < ...; row++) {
827
828     // TODO: Middle loop: iterate through shapes in REVERSE order (last index to first
829     //         ).
830     // Reason: Digits were stored least-significant-first.
831     // Reversing the order ensures they print from left to right (most significant
832     //         digit first).

```

```

831     for (int k = ...; ...; ...) {
832
833         // TODO: Inner loop: iterate over each column of the current shape.
834         for (int col = ...; col < ...; ...) {
835             std::cout << shapes[...].at(..., ...); // TODO: print character at (row, col)
836                 of shapes[k]
837         }
838     }
839
840     std::cout << '\n';
841 }
842
843 void display(const std::string& str) {
844     TurtleGraphics tg;
845     std::vector<Floor> shapes;
846
847     // TODO: Process each character ch in str:
848     // (1) Uppercase letter (isupper): run CMDS_CAP_LETTERS[ch - 'A'].
849     // Example: 'A' -> CMDS_CAP_LETTERS[0], 'B' -> CMDS_CAP_LETTERS[1].
850     // (2) Lowercase letter (islower): convert to uppercase with toupper(ch),
851     // then apply the same logic as (1).
852     // (3) Digit character (isdigit): run CMDS_DIGITS[ch - '0'].
853     // Example: '0' -> CMDS_DIGITS[0], '1' -> CMDS_DIGITS[1].
854     // Note: unlike display(int), where digits are extracted numerically,
855     // here a digit is a char (from '0' to '9'), so use ch - '0' to get its index.
856     // (4) Space character (isspace): call tg.restart() to produce a blank floor.
857     // (5) Any other character: skip it with continue.
858     // (6) After processing ch, append tg.getFloor() to shapes.
859     // (7) Call tg.restart() to reset the floor and turtle for the next character.
860
861     if (...) { // TODO: condition: shapes is empty (str had no drawable characters)
862         std::cout << "no string inputed" << "\n";
863         return;
864     }
865
866     // Shapes are stored left-to-right, matching the order of characters in str.
867     // Example: "AB 12" -> shapes contains: [A, B, space, 1, 2]
868     const Floor& floor = ...; // TODO: first element of shapes
869
870     // TODO: Outer loop: iterate over each row of the floor.
871     for (int row = ...; row < ...; ...) {
872
873         // TODO: Middle loop: iterate over shapes in FORWARD order (first to last),
874         // since characters in str are processed left to right.
875         for (int k = ...; ...; ...) {

```

```

876 // TODO: Inner loop: iterate over each column of the current shape.
877 for (int col = ...; col < ...; ...) {
878     std::cout << shapes[...].at(..., ...); // TODO: print character at (row, col)
879         of shapes[k]
880 }
881 }
882
883 std::cout << '\n';
884 }
885 }

```

7.4 Compile and Run

Compile all files together with the following commands.

```
g++ -o draw Floor.cpp Turtle.cpp TurtleGraphics.cpp drawSideBySide.cpp
./draw
```

7.5 Sample Input and Output

If we input -21 and “CS 135” (without quotes) respectively, here are a sample output (I omit the output of each individual character).

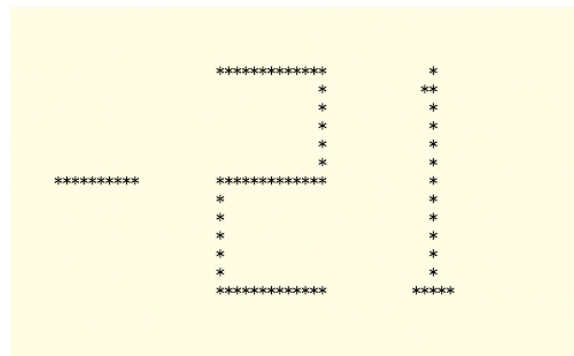


Figure 1: Display -21

7.6 Submission for Task D

Submit `drawSideBySide.cpp` to gradescope.

8 Makefile: a Tool for Project Build Management

Compiling and linking multiple source files into an executable can be time-consuming and error-prone. Using a Makefile simplifies this process.

Download link: [Makefile](#). Its contents are as follows.

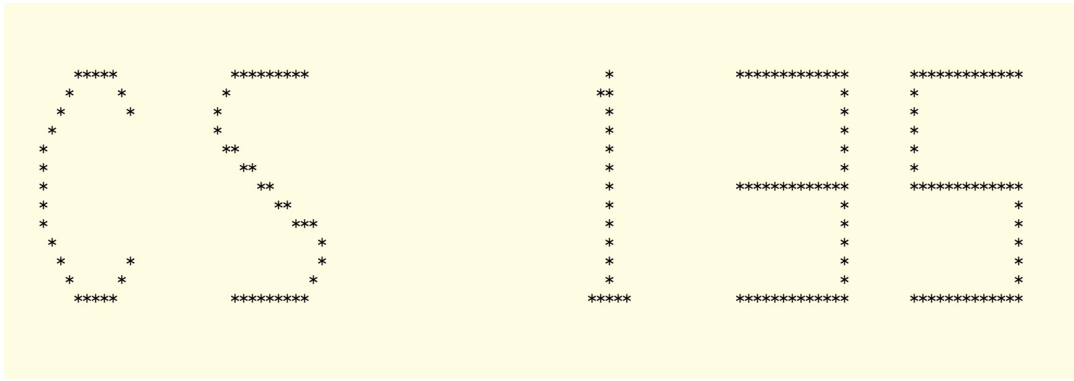


Figure 2: Display “CS 135”

```

1 # Compiler
2 CXX = g++
3 CXXFLAGS = -std=c++17 -Wall -Wextra -pedantic
4
5 # Object files
6 FLOOR_OBJ = Floor.o
7 TURTLE_OBJ = Turtle.o
8 TG_OBJ = TurtleGraphics.o
9
10 # -----
11 # Default target
12 # -----
13 all: floor turtle tg draw
14
15 # -----
16 # Task A: Floor only
17 # -----
18 floor: FloorUnitTest.o $(FLOOR_OBJ)
19     $(CXX) $(CXXFLAGS) -o floor FloorUnitTest.o $(FLOOR_OBJ)
20
21 # -----
22 # Task B: Turtle only
23 # -----
24 turtle: TurtleUnitTest.o $(TURTLE_OBJ)
25     $(CXX) $(CXXFLAGS) -o turtle TurtleUnitTest.o $(TURTLE_OBJ)
26
27 # -----
28 # Task C: TurtleGraphics
29 # -----
30 tg: TurtleGraphicsUnitTest.o $(TG_OBJ) $(TURTLE_OBJ) $(FLOOR_OBJ)
31     $(CXX) $(CXXFLAGS) -o tg TurtleGraphicsUnitTest.o $(TG_OBJ) $(TURTLE_OBJ) $(
        FLOOR_OBJ)

```

```

32
33 # -----
34 # Draw program (depends on ALL)
35 # -----
36 draw: drawSideBySide.o $(TG_OBJ) $(TURTLE_OBJ) $(FLOOR_OBJ)
37     $(CXX) $(CXXFLAGS) -o draw drawSideBySide.o $(TG_OBJ) $(TURTLE_OBJ) $(FLOOR_OBJ)
38
39 # -----
40 # Compilation rules
41 # -----
42 Floor.o: Floor.cpp Floor.hpp
43     $(CXX) $(CXXFLAGS) -c Floor.cpp
44
45 Turtle.o: Turtle.cpp Turtle.hpp
46     $(CXX) $(CXXFLAGS) -c Turtle.cpp
47
48 TurtleGraphics.o: TurtleGraphics.cpp TurtleGraphics.hpp Turtle.hpp Floor.hpp
49     $(CXX) $(CXXFLAGS) -c TurtleGraphics.cpp
50
51 # Unit tests
52 FloorUnitTest.o: FloorUnitTest.cpp Floor.hpp
53     $(CXX) $(CXXFLAGS) -c FloorUnitTest.cpp
54
55 TurtleUnitTest.o: TurtleUnitTest.cpp Turtle.hpp
56     $(CXX) $(CXXFLAGS) -c TurtleUnitTest.cpp
57
58 TurtleGraphicsUnitTest.o: TurtleGraphicsUnitTest.cpp TurtleGraphics.hpp Turtle.hpp
59     Floor.hpp
60     $(CXX) $(CXXFLAGS) -c TurtleGraphicsUnitTest.cpp
61
62 # Draw program
63 drawSideBySide.o: drawSideBySide.cpp TurtleGraphics.hpp Turtle.hpp Floor.hpp
64     $(CXX) $(CXXFLAGS) -c drawSideBySide.cpp
65
66 # -----
67 # Clean
68 # -----
69 clean:
70     rm -f *.o floor turtle tg draw

```

To compile and link Floor.cpp and FloorUnitTest.cpp in Task A, run

```
make floor
```

Then test the default constructor Floor class using

```
./floor A
```

To compile and link Turtle.cpp and TurtleUnitTest.cpp in Task B, run

```
make turtle
```

Test reset method of Turtle class using

```
./turtle A
```

To compile and link Floor.cpp, Turtle.cpp, TurtleGraphics.cpp, and TurtleGraphicsUnitTest.cpp in Task C, run

```
make tg
```

Then test default constructor of TurtleGraphics class using

```
./tg A
```

To compile and link Floor.cpp, Turtle.cpp, TurtleGraphics.cpp, and drawSideBySide.cpp in Task D, run

```
make draw
```

or simply

```
make
```

Then run the executable file using

```
./draw
```

Call the following command to clean all generated object files and executable files.

```
make clean
```